

# @SKILLS: ATTENTION IS ALL YOU HAVE

An open agent skills protocol, built for the future of knowledge sharing

A PREPRINT

Li Yin<sup>1</sup> Zhi Li<sup>1</sup> Zhan Shi<sup>1</sup> Haoran Zhang<sup>1</sup> Haebin Seong<sup>1</sup> Zhangyang (Atlas) Wang<sup>2\*</sup>  
<sup>1</sup>SylphAI <sup>2</sup>The University of Texas at Austin

## ABSTRACT

Agent skills package procedural knowledge as SKILL.md files: 56,804 are published today, and teams write many more privately. The dominant way to deliver one is to install it, after which its description sits permanently in the system prompt, competing for fewer than a hundred reliable slots in which the model may or may not match it to a request. So the long tail has no path to use, and a team’s own playbooks compete with everything else for the same scarce room. Our observation is that installing bundles three separable functions—content, persistence, and auto-triggering—and only the last needs the prompt. We propose @skills, an open protocol that separates them. A path addresses any skill, any subtree, or a whole collection, and reading it is using it, so nothing installs and nothing becomes resident; `:save` vendors a copy at that same path into the project’s git-tracked tree, to adapt and own it; `:install` adds one .gitignore-style line, which is the only thing that costs prompt residency. A directory is a menu, so a “bundle” is just a directory and all-or-nothing delivery does not exist. There is no manifest, lockfile, or registration, because a file tree needs none, and SKILL.md is unchanged. The protocol is additive, ships as an installable package with an open specification (<https://github.com/SylphAI-Inc/atskills>), and turns any agent that can read files and run commands into a client from a single instruction file. The protocol is implemented in the AdaL CLI (<https://adalagent.ai>). Because a file tree addresses skills well but cannot find them, the protocol also pairs with a free agent skills hub (<https://atskills.one>) confined to the jobs paths cannot do: search and ranking across the whole public corpus, hosting for skills that have no repository, private and team collections, and one-screen authoring for the non-developers whose procedural knowledge skills capture best. The hub is a service and never a requirement—gh: and local paths resolve with no hub involvement, and GitHub-hosted skills keep their gh: identity even when the hub indexes them. Install less, use more.

**Keywords** AI agents · agent skills · prompt engineering · context management · tool use · open protocols

## 1 Introduction

Coding agents increasingly extend themselves with *skills*: packages of procedural knowledge, such as how to produce a changelog, deploy a service, or typeset a paper, written as Markdown instructions in a standardized SKILL.md file. Our July 2026 crawl of the public ecosystem finds 56,804 such skills across 1,133 GitHub repositories, indexed by community registries such as skills.sh [1]. The official definition says what a skill is:

*A skill is a directory containing a SKILL.md file: organized folders of instructions, scripts, and resources that give agents additional capabilities.* —the official definition [2, 3]

What the published corpus shows is what skills have become *for*, and this paper’s argument rests on that shift, so we extend the definition (evidence in Appendix B):

\*The open-source protocol—specification, agent integration guide, and example workflows—is at <https://github.com/SylphAI-Inc/atskills>. Correspondence via the repository’s issue tracker.

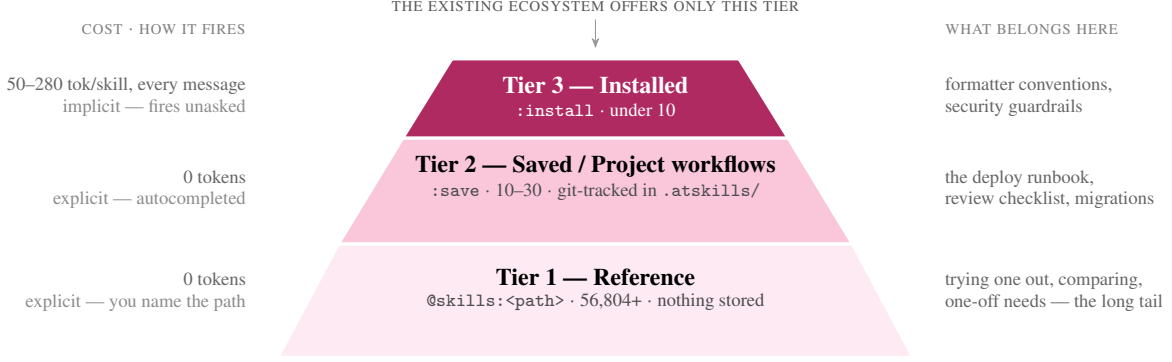


Figure 1: The three delivery tiers, sized by how many skills each is for. A skill belongs in the *cheapest tier that meets its activation need*, and only deliberate promotion moves it up—so nothing persists by accident. Tiers 1 and 2 differ solely in where the bytes live: nowhere, or vendored in the project’s git-tracked tree, where the @ autocomplete indexes them in the tooling rather than in the prompt. Neither costs a resident token, and both fire explicitly, by name, at the point of use. Tier 3 alone pays residency, and buys exactly one thing with it—firing without being asked—for its frontmatter alone. The existing ecosystem offers only that apex, so all 56,804 skills compete for an attention budget argued in Appendix C to be well under a hundred slots per agent.

*A skill is procedural knowledge for an AI agent, either the world’s expertise or a team’s own way of doing things, and it serves three roles: domain knowledge, expertise distilled into instructions; provider integration, by which a service teaches any agent to drive its product; and team workflows, the customized procedures a team adapts to its own way of working. A model knows what the world knows; a skill adds what it doesn’t.*

Skills spread because they are simpler than what came before. The Model Context Protocol [4] standardized agent-to-tool connectivity but is heavyweight on both sides, since the provider maintains a server and the user pays for resident tool schemas before any work begins, whereas a skill is a folder of text the agent simply reads [5]. Yet installed skills kept MCP’s delivery model in miniature: something is still installed per machine, and just as MCP loads every server’s tool definitions at session start, an installed skill puts its description into the system prompt in every session, relevant or not. The artifact got radically simpler; the *residency* did not.

That inheritance is the subject of this paper. Installation is what an agent skill’s description buys with permanent space in the prompt, and the only thing it buys is *auto-triggering*—firing without being asked. Auto-triggering is also scarce, with a reliable capacity well under a hundred slots per agent, so 56,804 published skills compete for fewer than a hundred (Sec. 2). Everything else installation carries—fetching the content, and keeping it for next time—needs no prompt space at all.

We propose @skills, which separates those three functions and lets each be decided on its own (Figs. 1 and 2). A path addresses any skill, any subtree, or a whole collection and is read at the point of use; :save keeps a copy in the project’s git-tracked tree; :install adds one line to a .gitignore-style file, the only thing that costs residency (Sec. 3). The management model is nothing but the file tree, so there is no manifest, lockfile, or registration, and SKILL.md is untouched.

**Terminology.** A *skill* is a directory, not just a Markdown file: SKILL.md holds frontmatter with a name and a one-line description followed by instructions, alongside optional scripts and reference files that load only when needed. A *plugin* is an installable bundle and a *marketplace* is a registry of bundles, though the two terms have no shared definition across agents (Appendix A); both inherit the install lifecycle. We call text *resident* when it occupies the prompt on every message of a session. The body of this paper carries the argument; the evidence, mechanics, and full protocol are in the appendices, loaded on reference.

## 2 The Problem: Attention Is a Limited Budget, and Installation Spends It

Installing a skill spends a slot in the resident prompt, and that budget is physically small: our evidence bounds it conservatively at fewer than a hundred reliable auto-trigger slots per agent. Yet installation is the only *persistent* way any of 56,804 published skills can reach an agent. A distribution channel is supposed to scale with the corpus, whereas

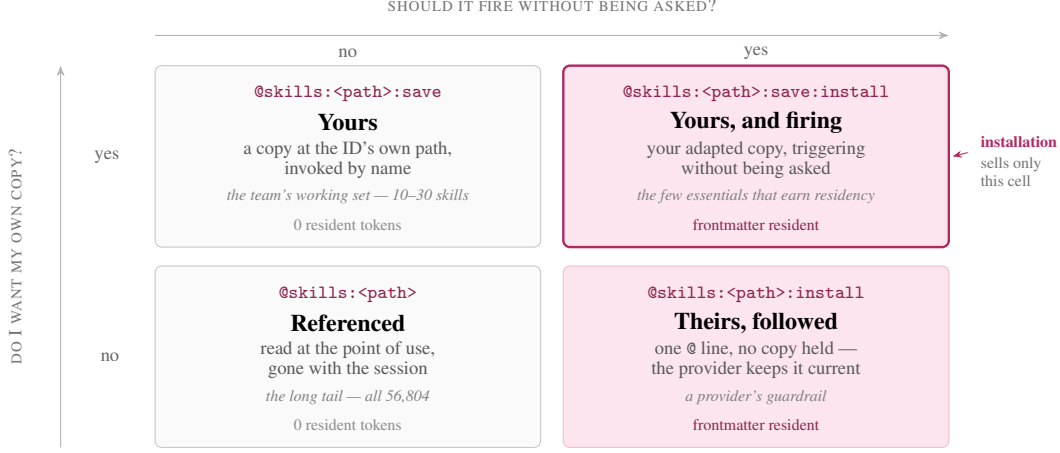


Figure 2: The contribution in one picture: installation bundles three functions, and the protocol separates them, so that what was a single switch becomes two independent decisions. **Left to right** is whether the skill should fire without being asked, which is the only thing that requires prompt residency; **bottom to top** is whether the project keeps a copy of its own. The two suffixes set these separately—`:install` writes one `.gitignore`-style line, `:save` vendors a copy at the ID's own path—and neither implies the other, so all four cells are reachable and each is right somewhere. Installing offers only the upper right, and offers it as the sole way to obtain content at all, which is why every one of 56,804 published skills has had to bid for an attention budget of fewer than a hundred slots in order to be used even once. Reading a skill costs nothing that persists, keeping one costs a folder in the repository, and only firing unprompted costs the prompt.

an attention budget cannot, because it is capped by the model's ability to attend. Everything in this section follows from that mismatch:

*56,804 indexed skills compete for fewer than 100 reliable auto-trigger slots per agent.*

The rest of this section is the evidence, at three levels: the cap is real (Level 1), the parties adapt to it in ways that consume more of it (Level 2), and because install is also the only *lifecycle*, the ecosystem inherits structural damage that no amount of budget would fix (Level 3). Nothing here indicts auto-triggering itself, which is genuinely valuable and which the tier model keeps for the instructions that need it; the problem is that one mechanism carries everything. Appendix C develops all six mechanisms in full.

**Level 1 — the cap is real.** Three properties of resident context bound what it can carry. *Distance decay*: a resident description sits at a fixed place, the top of the context, while every turn, file read, and tool result lands between it and the current request; adherence to the system message measurably decays over turns [6–8]. *The standing tax*: each installed description is paid on every message, relevant or not—50–280 tokens per skill in our measurements—and input length alone degrades reasoning well below the nominal context limit [9, 10]. *Dilution*: each added description competes with the rest, and instruction-following degrades systematically as concurrent instructions accumulate [11, 12]. Bundling amplifies all three, since the dominant harness offers no per-skill control inside a plugin [13]. Together they yield the bound above (Sec. C.1).

**Level 2 — the adaptations make it worse.** A scarce channel is one every party works around, and each workaround consumes more of what is scarce.

*Authors bid for slots.* A skill fires only if the model matches the user's message against its one-line description, and a miss is silent—the user never learns the skill was there. So authors stop writing descriptions and start writing bids, padding that one line with trigger and skip conditions: *read this BEFORE opening the file; don't skip because it looks like a one-liner; trigger whenever the prompt mentions ...; skip only when ...*. Stripe's flagship skill carries roughly 150 words of them. Rational individually, ruinous collectively: such descriptions cost roughly  $20\times$  the tokens of a quiet one, so every bid deepens the tax and the dilution just measured (Sec. C.2).

*Users install less than they could.* A permanent cost is worth paying only for a skill used often, genuinely better fired unprompted, and known to trigger reliably. Trying something out, comparing alternatives, and one-off needs all fail at least one bar while paying full price—permanence, plus management in every agent and on every machine. The rational move is not to bother, which is how most of the corpus goes unexplored.

*What is installed gets forgotten.* Installing is the easy part. The skill then sits in a hidden directory and is supposed to fire on its own, so the user stops tracking it. When it fails to fire, nothing says so—and they cannot invoke it by name either, because they no longer remember it is there.

*Teams skip the channel entirely,* pasting their workflows into a monolithic AGENTS.md: always loaded, zero setup, no trigger lottery. The crudest mechanism wins on the only axis users feel—simplicity.

**Level 3 — the damage outlasts the budget.** Suppose the slots were free tomorrow. Three failures would remain, because they come from install being the only *lifecycle* rather than from the cap. First, everything lands in one flat bin: a skill grabbed once for a one-off task sits forever beside the playbooks a team runs daily, indistinguishable from them, so the procedures that matter most have no dedicated home. Second, no shared vocabulary exists to build one from—the layer above the skill, the “plugin,” *contains* skills in one agent, sits *beside* them as a sibling category in another, and is *one of seven* peer extension types in a third [14–16]—so each agent ships its own hierarchy and its own inventory screen over what are, on disk, the same Markdown directories. Third, none of that machinery serves private work: it grew out of publishing, aimed at distributing skills to strangers, and a team whose skills will never leave the company is served by none of it (Appendix A).

The long tail therefore has no delivery path, authors see no measurable usage and stop publishing, and users conclude the ecosystem is thin. The install lifecycle, not the file format, is what strangles it.

### 3 The @skills Protocol

**The design principle: three tiers.** Every skill currently pays the same maximal cost, permanent residency, for the same scarce benefit, a chance at auto-triggering, whether it is used once a year or on every message. The fix is to let each skill sit in the *cheapest tier that meets its activation need*, which is what separating content, persistence, and triggering makes possible: three tiers (Fig. 1), of which the ecosystem offers only the most expensive. Appendix D develops their storage and why the protocol defines no user-level install.

**What it is, and where to get it.** The protocol is four files and one surface: a reference command (@skills:<path>), a project folder (.atskills/), a trigger file (.autotrigger), and a provenance stamp (.source, two lines recording where a saved copy came from and at which revision), plus a management surface (/skills) that every client should ship. The unit of content is the unmodified SKILL.md directory, so this is a delivery layer rather than a fork of the format. All of it ships as one installable package, `npm i atskills`, holding the normative specification, a TypeScript core, a reference CLI, an agent instruction file, and a runnable demo [17]; none of what follows is a proposal, since it is implemented, tested, and in production in AdaL [18]. Appendix E gives the full specification and Appendix F the algorithms.

**Syntax.** A skill is referenced inline, by the user or by the agent itself. The path is the identity, and the two suffixes are orthogonal and combinable:

|                                 |                                              |
|---------------------------------|----------------------------------------------|
| @skills:deploy                  | the project’s own -- .atskills/deploy/       |
| @skills:hub:sylphai/glowmotion  | one skill, from the hub                      |
| @skills:gh:acme/skills/deploy   | one skill, straight from GitHub              |
| @skills:gh:stripe/agent-toolkit | a directory -> a menu, one line per skill    |
| @skills:<path>:save             | copy into the project -- to adapt it         |
| @skills:<path>:install          | one line in .autotrigger -- fires on its own |

This reuses the most widely adopted gesture in coding agents, since @ is already how users add files and directories in a dozen of them [19–22], and skills simply become one more addressable resource. A path with no SKILL.md is a directory rather than a failure, and it loads as a *menu*: one line per skill beneath it, each line itself a valid path. Browsing and using are therefore the same gesture, a collection can be taken subtree by subtree, and granularity belongs to the reader instead of whoever packaged the bundle.

**Composition.** Several references load in one message, five of them in Fig. 3, each injected at its own point of use, and they chain across a workflow’s steps. Because every reference is explicit, co-firing  $N$  skills is deterministic, whereas under installation it is a lottery in which  $N$  descriptions must each win a probabilistic match at once, with the odds compounding against exactly the multi-skill workflows real work is made of (Sec. C.1). Sources also mix freely within a message, so a workflow is no longer confined to what one publisher shipped together. This is the capability install-only delivery cannot reproduce at any budget, and the nearest non-installing tool in the ecosystem does not have it either, resolving exactly one skill per invocation and refusing any reference that matches more than one [1]. Composition has to be a property of the address, not of the fetch.

**Resolution: the prefix decides.** A reference states where it comes from, so resolution never guesses: a bare path is the project’s own and never reaches the network, while `hub:` and `gh:` name the cloud. Both cloud forms still resolve local-first, because a saved copy sits at its ID’s own path and therefore answers its own address, which is vendoring as in Go’s `vendor/` directory, and which is what makes the team’s adaptation the meaning of that address inside the project. Otherwise the content comes through one machine-wide validating cache (`~/ .cache/atskills`, required to sit outside any `.atskills/` directory) that behaves like a browser: every use asks the source whether anything changed, in a single revision probe rather than a re-download, so that unchanged serves instantly, changed fetches fresh, and offline serves the cached copy marked stale (Algorithm 1). No manifest is ever consulted, because none exists, and cache entries are always safe to delete.

**Ownership: theirs, or yours.** Existing install systems leave a skill in an ambiguous middle state, installed but stale, enabled but of unknown origin, or pending an update that may never be applied. The protocol admits two relationships and eliminates the third by construction. A skill is *theirs* when one `@` line follows the provider’s copy without holding one locally, which is the appropriate arrangement precisely because the provider is documenting a service they continue to change. It becomes *yours* when `:save` copies it to its ID’s own path and detaches it at that moment, so the copy is thereafter an ordinary file of the project rather than a subscription. This is also why there is no update lifecycle and no version pinning: frozen text against a moving service offers only the appearance of safety. Provenance is two lines in `.source`, written once and never consulted during resolution, and a user who wants a saved copy refreshed simply saves again, which replaces an unedited copy and refuses an edited one rather than overwriting work (Algorithm 2).

**Auto-triggering: one line in one file.** `.autotrigger` follows `.gitignore` conventions, one entry per line with `#` for comments, where a plain line matches the project’s own skills (so `team-flows/` triggers everything beneath it), an `@` line follows a cloud skill, and a trailing `/` takes a whole directory. Installing therefore consists of adding a line and uninstalling of removing one, and nothing further from the classic install lifecycle remains because nothing further proved necessary. What a line costs is the skill’s frontmatter alone, some 50–100 tokens held from session start, the body loading only when the skill fires (Algorithm 3). The suffixes, the `/skills` checkbox tree, and hand edits all write these same lines, and `/skills` adds a view no agent currently ships: the exact resident text the model will receive, with its token count.

**What the separation buys.** Because the two suffixes are independent, a user answers two questions instead of one, whether to own a copy and whether the skill should fire unprompted, where installation poses a single bundled question and answers it with content, persistence, and triggering together. All four combinations are then reachable and each is right somewhere (Fig. 2). Two further properties follow. Reversal is symmetric, since uninstalling deletes the line that installing added and nothing else was created to undo, and everything stays reviewable, since what fires unprompted is a single file of one-line diffs while what the team owns is a folder in the repository. Together these turn the cost of a skill from a default into a decision, which is what allows 56,804 skills to be reachable while fewer than ten are resident.

**Adoption: one file, or one CLI.** The protocol asks nothing of the agent or its vendor. `SKILLS.md`, a single instruction file, turns any agent that can read files, run shell commands, and fetch URLs into a full client, resolution and cache and save and trigger rules included; agents that prefer to shell out call the reference CLI (`atskills get / save / triggers / prompt`) and inherit the same behavior without implementing it. Either way a resolved skill is just files on disk, so agents that load skills their own way keep working untouched. Native `@` integration stays small because it reuses the `@` context system every modern agent already has, and the host needs no protocol logic at all: the client computes the resident prompt block and the host splices in one string (Appendix E).

**The hub.** A file tree addresses skills well but cannot find them, host what has no repository, or show anyone what they have, so the protocol pairs with a free hub (<https://atskills.one>) whose value is deliberately confined to exactly those jobs. It *finds*: search and ranking across the whole public corpus, which no amount of path grammar provides, with every result a copyable `@skills:` reference. It *hosts*: a skill authored on the hub needs no GitHub repository at all, which matters because publishing today effectively requires maintaining one, and that keeps authorship technical even though the artifact is plain Markdown anyone could write. Private and team collections are hosted the same way, so work that will never be published gets the same addressing, sharing, and portability as work that is—and a personal collection travels with the user across projects, machines, and agents rather than living in a directory on one laptop. It *manages*: a visual library of what a person or team owns, with usage visible per skill, where the file tree can only show a project one folder at a time. And it *authors*: one screen, no repository, no git, for the non-developers whose procedural knowledge skills capture best. The division of labor is that GitHub hosts, the hub finds, and `@` uses. The git and GitHub precedent is the governing answer to why the hub can never be required: `gh:` paths and local folders resolve with zero hub involvement, forever, and GitHub-hosted skills keep their `gh:` identity even when the hub indexes them, since only skills authored on the hub carry its name. The hub earns its place on management value rather than lock-in, which is the same deal GitHub took.



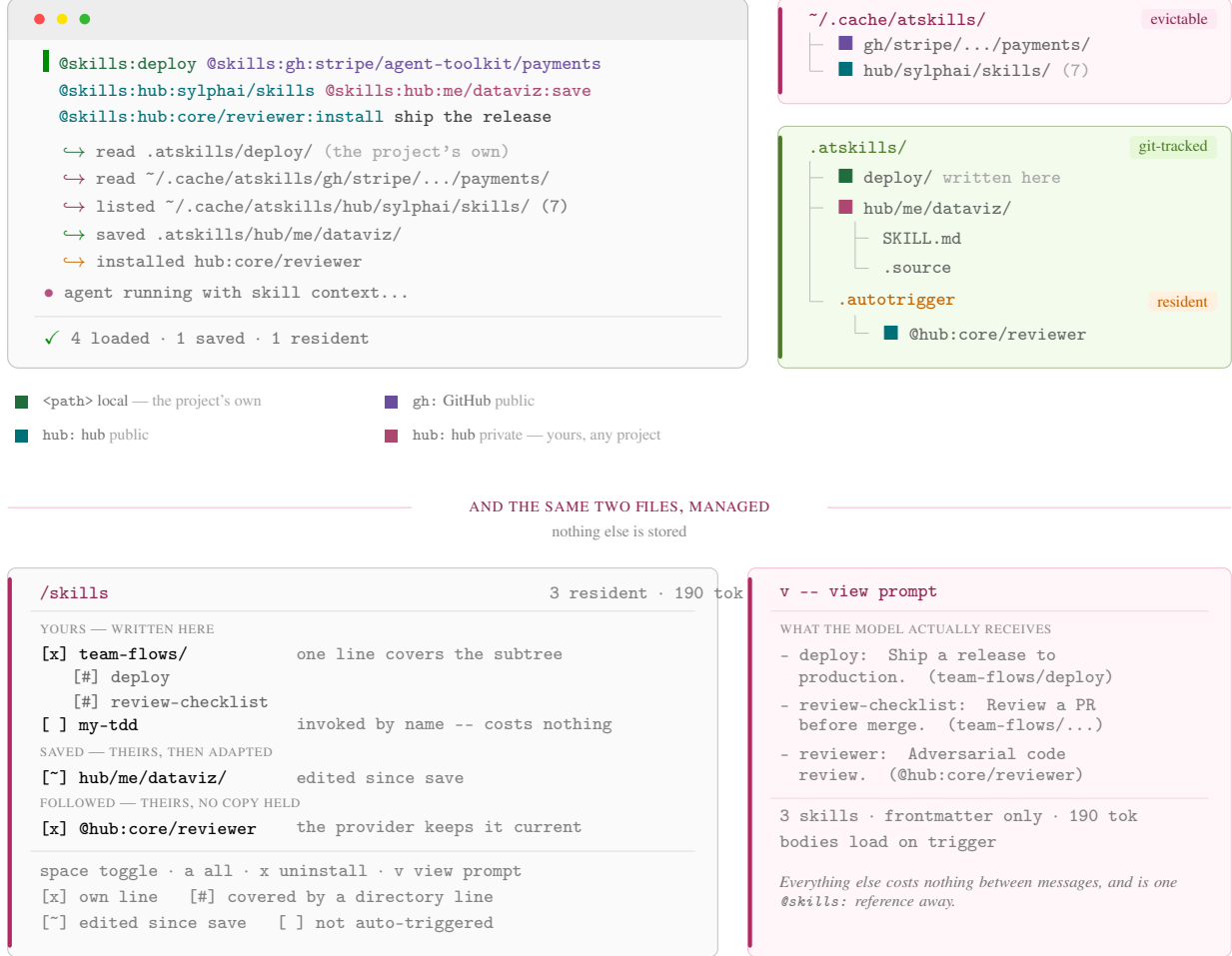


Figure 3: The whole protocol in one look (AdaL [18]; reference client [17]). **Above, using it.** Five references in one message, spanning four sources—the project’s own skill by a bare path, a public GitHub repository, the public hub, and the user’s private hub collection—and four jobs: three skills read, one of them a *directory* loaded as a menu of its seven; one :*saved*; one :*installed*. One grammar covers all four sources, so a private skill is referenced, saved, and triggered exactly like a public one and travels with the user rather than with any codebase, and resolution is local-first, which is why a bare path and a saved copy answer identically. To the right is everything the message produced: an evictable cache entry for what was merely used, a copy at the ID’s own path for what was saved, and one line for the only skill that will fire unprompted. **Below, managing it.** The same two files, made visible. `/skills` is a checkbox tree in which `[x]` is a skill’s own line, `[#]` means a directory line above it already covers it, and `[~]` marks a saved copy edited since it was taken; unchecking under a covering line splits that line into explicit lines for the siblings that stay on, so the file always reads true. The surface holds no state beyond those two files, which is why hand-editing them, typing the suffixes, and clicking here are the same operation. *View prompt* then shows the exact resident text, verbatim, with its token count—the view no agent ships today, and what turns a probabilistic trigger budget into a number a team can see and review as a diff.

## 4 Discussion and Conclusion

**The fix is subtraction, not addition.** The most useful thing to say about this protocol is how little of it is new. It adds no field to `SKILL.md`, no manifest, no lockfile, no database, and no service anyone is required to use. Nearly every piece already existed and is simply being used for delivery: the skill directory as the unit of content, the @ gesture users already type to add files, `.gitignore` semantics for saying what fires on its own, git for transport and for revisions, vendoring for why a saved copy answers its own address, and the filesystem itself as the manager. What the protocol mostly does is take things away—the install lifecycle, the update command, version pinning, per-agent directories, and the whole packaging layer above the skill—and what remains is a path, a folder, and a file of one-line decisions. A user learns one character. An agent builder adds one file, or one dependency.

That matters because the competing convention won on exactly this axis. `AGENTS.md` became dominant not by being better written but by being trivial to deliver, and anything meant to succeed it has to be at least as simple to use, not merely better designed (Appendix G).

**What it changes for the ecosystem.** For users, a skill can be tried once without being paid for on every message afterwards, so the decision to reach for one stops being a commitment. For authors, this is the difference between album sales and plays: today a skill outside a user’s small installed set is never installed, so the long tail measures zero and its authors stop publishing, whereas usage counted per reference gives every published skill a real chance of being used. For teams, private work finally gets what public work has—one address, one place, reviewed like code and arriving with `git clone`—rather than an ecosystem built entirely for distributing to strangers. And for agent builders, a shared, agent-neutral store means a team stops maintaining the same skills once per agent, which is the cost that made working across agents impractical.

**Limits.** The protocol is additive, so installs, plugins, and vendor directories keep working, and nothing here asks a user to migrate. Our central quantity, the number of reliable auto-trigger slots, is bounded by argument and by the literature rather than measured by us, and the corpus figures come from one crawl and one working setup. On security, a referenced skill is remote text an agent will act on, which is the channel indirect prompt injection uses [23, 24]; the answer is provenance and review rather than filtering, and the tiers form a trust ladder from a single throwaway session, to a git diff on save, to a one-line diff before anything joins the resident prompt. The risk is not new—an installed skill’s body loads invisibly—but it is not removed either.

**Future work.** The measurements this argument invites are trigger reliability as a function of installed-skill count, and compliance as a function of injection position, both for skill-sized instructions; alongside them, the full classification of all 56,804 skills that Appendix B begins, and reference-based usage counting in a public catalog, which is the direct test of whether authors publish more when plays rather than installs are the measure of use.

The principle generalizes past skills: *resident context is a budget; spend it only on what must fire implicitly, and deliver everything else at the point of use, where attention is highest.*

*Install less, use more.*

## References

- [1] Vercel. skills.sh: The agent skills registry. <https://skills.sh>, 2025. Community registry and CLI (npx skills) for agent skills. Behavioural claims in this paper refer to the open-source CLI at <https://github.com/vercel-labs/skills>, accessed 2026-08-04, which provides add (install), use (one skill, no install), find, update, and remove.
- [2] Anthropic. Agent skills. <https://www.anthropic.com/news/skills>, 2025. Open format for packaging procedural knowledge for AI agents.
- [3] Anthropic. Equipping agents for the real world with agent skills. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>, 2025. Anthropic Engineering blog, October 16, 2025.
- [4] Anthropic. Introducing the model context protocol. <https://www.anthropic.com/news/model-context-protocol>, 2024.
- [5] Simon Willison. Claude skills are awesome, maybe a bigger deal than MCP. <https://simonwillison.net/2025/Oct/16/claude-skills/>, 2025. Blog post, October 16, 2025.

- [6] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [7] Yanzhao Qin, Tao Zhang, Yanjun Shen, et al. SysBench: Can large language models follow system messages?, 2024. arXiv:2408.10943.
- [8] Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. LLMs get lost in multi-turn conversation, 2025. arXiv:2505.06120.
- [9] Mosh Levy, Alon Jacoby, and Yoav Goldberg. Same task, more tokens: the impact of input length on the reasoning performance of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [10] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekeshe, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? In *First Conference on Language Modeling (COLM)*, 2024.
- [11] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- [12] Daniel Jaroslawicz, Brendan Whiting, Parth Shah, and Karime Maamari. How many instructions can LLMs follow at once?, 2025. arXiv:2507.11538.
- [13] Anthropic. Claude code documentation: Discover and manage plugins. <https://code.claude.com/docs/en/discover-plugins>, 2026. Plugin install, enable, and disable operate at whole-plugin granularity.
- [14] Anthropic. Claude code documentation: Plugins reference. <https://code.claude.com/docs/en/plugins-reference>, 2026. A plugin bundles skills, subagents, hooks, and MCP servers into one installable directory.
- [15] OpenAI. Plugins in ChatGPT and Codex. <https://help.openai.com/en/articles/20001256>, 2026. A plugin is an installable bundle that may contain skills, app connectors, or both.
- [16] Cursor. Cursor documentation: Skills. <https://cursor.com/docs/context/skills>, 2026. Skills appear alongside plugins, MCPs, subagents, rules, commands, and hooks as parallel extension types.
- [17] SylphAI. atskills: The @skills protocol — specification, reference implementation, and agent integration guide. <https://github.com/SylphAI-Inc/atskills>, 2026. Normative specification (PROTOCOL.md), agent instruction file (SKILLS.md), reference CLI and TypeScript core, and a runnable demo. Accessed 2026-08-04.
- [18] SylphAI. AdaL: The AI engineer agent. <https://adalagent.ai>, 2026.
- [19] Cursor. Cursor documentation: @ symbols. <https://docs.cursor.com/context/@-symbols/overview>, 2025.
- [20] GitHub. Using GitHub copilot chat: Chat participants and context variables. <https://docs.github.com/en/copilot/using-github-copilot/copilot-chat>, 2024.
- [21] Cline Bot Inc. Adding context: @-mentions — cline documentation. <https://docs.cline.bot/core-workflows/working-with-files>, 2026.
- [22] Sourcegraph, Inc. Cody chat: @-mentions and context — sourcegraph documentation. <https://sourcegraph.com/docs/cody/capabilities/chat>, 2026.
- [23] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, et al. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.
- [24] Yupei Liu, Yuqi Jia, Runpeng Geng, et al. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*, 2024.
- [25] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, et al. Model context protocol (MCP) at first glance: Studying the security and maintainability of MCP servers, 2025. arXiv:2506.13538.
- [26] Anthropic. Code execution with MCP: Building more efficient AI agents. <https://www.anthropic.com/engineering/code-execution-with-mcp>, 2025. Anthropic Engineering blog, November 2025.
- [27] Daniel Miessler. Anthropic changes MCP calls into filesystem-based skills. <https://danielmiessler.com/blog/anthropic-downplays-mcps>, 2025. Blog post, November 5, 2025.
- [28] Peter Kellner. Are MCP servers going obsolete? skills vs MCP. <https://peterkellner.net/2026-03-10-are-mcp-servers-going-obsolete-skills-vs-mcp/>, 2026. Blog post, March 10, 2026.



- [29] Cloudflare. Cloudflare skills: the wrangler skill. <https://github.com/cloudflare/skills>, 2026.
- [30] Expo. Expo skills. <https://github.com/expo/skills>, 2026.
- [31] Stripe. Stripe agent skills. <https://docs.stripe.com/skills>, 2026.
- [32] Sentry. Agentic usage — sentry CLI documentation. <https://cli.sentry.dev/agentic-usage/>, 2026.
- [33] Addy Osmani. Agent skills. <https://www.oreilly.com/radar/agent-skills/>, 2026. O’Reilly Radar, May 27, 2026.
- [34] Agent Skills Community. Agent skills discovery specification. <https://agentskills.io>, 2025. .well-known/agent-skills self-hosted discovery convention, v0.2.0.
- [35] OpenAI and contributors. AGENTS.md: A simple, open format for guiding coding agents. <https://agents.md>, 2025.
- [36] Xiaobo Guo and Soroush Vosoughi. Serial position effects of large language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.
- [37] Cheng-Yu Hsieh, Yung-Sung Chuang, Chun-Liang Li, Zifeng Wang, Long T. Le, Abhishek Kumar, James Glass, Alexander Ratner, Chen-Yu Lee, Ranjay Krishna, and Tomas Pfister. Found in the middle: Calibrating positional attention bias improves long context utilization. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.
- [38] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, et al. LLMingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [39] Anthropic. Effective context engineering for AI agents. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>, 2025. Anthropic Engineering blog, September 29, 2025.
- [40] SylphAI. The agent skills hub: Search and visual management for @skills. <https://atskills.one>, 2026. Search over the public corpus, copyable @skills: references, collection browsing, and private/team collections. Never required: gh: paths resolve with no hub involvement.
- [41] Simon Willison. Prompt injection attacks against GPT-3. <https://simonwillison.net/2022/Sep/12/prompt-injection/>, 2022. Blog post, September 12, 2022.
- [42] Invariant Labs. MCP security notification: Tool poisoning attacks. <https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks>, 2025.
- [43] Umar Iqbal, Tadayoshi Kohno, and Franziska Roesner. LLM platform security: Applying a systematic evaluation framework to OpenAI’s ChatGPT plugins. In *Proceedings of the 7th AAAI/ACM Conference on AI, Ethics, and Society (AIES)*, 2024.
- [44] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (MCP): Landscape, security threats, and future research directions. *arXiv preprint arXiv:2503.23278*, 2025.
- [45] Yao Lu, Max Bartolo, Alastair Moore, et al. Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.
- [46] Tony Z. Zhao, Eric Wallace, Shi Feng, et al. Calibrate before use: Improving few-shot performance of language models. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.
- [47] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [48] Yunfan Gao, Yun Xiong, Xinyu Gao, et al. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- [49] Ohad Rubin, Jonathan Herzig, and Jonathan Berant. Learning to retrieve prompts for in-context learning. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2022.
- [50] Akari Asai, Zeqiu Wu, Yizhong Wang, et al. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024.
- [51] Lilian Weng. LLM powered autonomous agents. <https://lilianweng.github.io/posts/2023-06-23-agent/>, 2023. Blog post, June 23, 2023.

- [52] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, et al. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024.
- [53] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.
- [54] Andrew Zhao, Daniel Huang, Quentin Xu, et al. ExpeL: LLM agents are experiential learners. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence (AAAI)*, 2024.
- [55] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.
- [56] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [57] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- [58] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024.
- [59] Yujia Qin, Shihao Liang, Yining Ye, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.
- [60] Carlos E. Jimenez, John Yang, Alexander Wettig, et al. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.
- [61] John Yang, Carlos E. Jimenez, Alexander Wettig, et al. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024.
- [62] Andrej Karpathy. Software is changing (again). <https://www.youtube.com/watch?v=LCemiRjPEtQ>, 2025. Keynote, Y Combinator AI Startup School, June 17, 2025.
- [63] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Anastasios N. Angelopoulos, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [64] Continue Dev, Inc. Context providers — continue documentation. <https://docs.continue.dev/customize/deep-dives/custom-providers>, 2026.

## A Background: The Agent Skills Ecosystem

*A skill is a directory containing a SKILL.md file: organized folders of instructions, scripts, and resources that give agents additional capabilities.* —the official definition [2, 3]

**From MCP to skills.** To see why skills spread so fast, and where their delivery model still falls short, start with what preceded them. The Model Context Protocol [4] standardized how agents connect to tools, as a client-server JSON-RPC protocol, and won near-universal support within six months of its November 2024 release, adopted by OpenAI and Google alike. But MCP is heavyweight on both sides. The provider must build and maintain a server against a full specification covering transports, authentication, tools, resources, and prompts, and the measured quality of third-party servers is poor: in a study of 1,899 open-source MCP servers, 66% exhibited code smells and 5.5% shipped tool-poisoning vulnerabilities [25]. Any open contribution ecosystem shares that hazard, and Appendix G discusses the skills analogue. The user pays a standing cost per integration: a separately configured server process, tool schemas resident in context before any work begins, and every intermediate tool result passing through the model. GitHub’s official server alone consumes tens of thousands of tokens across its ~93 tool definitions [5]. By November 2025, MCP’s own steward was steering integrations away from direct tool calls and toward code execution over the filesystem, reporting a drop from 150,000 to 2,000 tokens (98.7%) on an equivalent task, with SKILL.md files as the durable, reusable layer [26, 27]. A skill, by contrast, is a folder of text the agent simply reads: no server, no protocol machinery, a fraction of the token cost [5]. Providers followed the simpler artifact [28]. Rather than maintain an MCP server, a provider ships a CLI plus a skill that teaches any agent to drive it. Cloudflare (wrangler), Expo (eas), Stripe, and Sentry all do this today [29–32]: one artifact, universal because virtually every coding agent has a shell.

**The SKILL.md format.** A skill is a directory containing a SKILL.md file: YAML frontmatter with a name and a one-line description, followed by Markdown instructions, plus optional scripts and reference files [2, 3]. The format

is an open standard and deliberately minimal. A skill is readable by humans and by any model without tooling, and that simplicity is widely credited for its rapid uptake, since the same file works across harnesses with no protocol machinery at all [5, 33]. Within a single loaded skill, the format already practices progressive disclosure: metadata stays resident, the body loads on relevance, and bundled files load on demand [3]. This paper extends that principle from *within one installed skill* to *delivery across the whole ecosystem*.

**What skills now carry.** The format was introduced for capabilities an agent lacked, but what people put in it has broadened, along two axes of ownership. *Publicly*, a skill is either domain expertise offered to anyone, or a provider’s integration path into every agent at once: rather than maintain a server per client, Cloudflare, Expo, Stripe, and Sentry each ship a CLI and a skill that teaches any agent to drive it [29–32]. *Privately*, a team encodes its own way of working, which is knowledge that is valuable precisely because it is not public and will never reach a market. The two carry different value and demand different handling, and the second is the case existing tooling was never built for: a registry ranks by install count, and a team’s deploy checklist has no install count. Appendix C shows what fills the gap instead.

**Installation.** Agents take on skills by copying them into an agent-specific directory such as `.claude/skills/`, `.windsurf/skills/`, or `.adal/skills/` in AdaL, our own agent (Appendix E). Across the 75 agents the community skills CLI supports there are 54 distinct project-level locations for the same format (Appendix B). Users can install and uninstall, but each operation costs two to four CLI commands or app clicks, once per agent and once per machine. That overhead defeats what skills are for: knowledge that is simply *there* when needed. At session start the agent puts each installed skill’s name and description into the system prompt, and the model is expected to match requests against those descriptions and load the full body when one seems relevant. We call this *auto-triggering*. The description is resident on every message; the body loads only on a trigger (Fig. 4).

**No shared terminology, and management to match.** Above the skill sits a packaging layer, and there is no agreed definition of what it contains. In Claude Code, a plugin is a directory that bundles skills together with subagents, hooks, and MCP servers, and a marketplace is a catalog of plugins, so skills are *children* of plugins: the desktop application lists installed plugins with a count of the skills inside each, and its composer reaches individual skills only through the plugin submenu [13, 14]. In the Codex application, plugins and skills are *siblings*: a single settings pane titled “manage plugins, skills, and MCPs” presents plugins, apps, MCP servers, and skills as four parallel lists, and a plugin there is an installable bundle that may contain skills, service connectors, or both [15]. In Cursor, skills are one of seven peer categories, alongside plugins, MCP servers, subagents, rules, commands, and hooks [16]. The same word denotes a different object in each product, and the nesting is inverted between two of them. The only term that means the same thing everywhere is the skill itself.

Management inherits the confusion. Every agent renders its own inventory through its own interface, over its own hierarchy, in its own vocabulary, so a user running three agents learns three mental models and three management surfaces for what are, on disk, the same Markdown directories. Some surfaces are thin: an agent may show which bundles are installed without offering a view of the individual skills inside them, or a way to act on one. And because this machinery grew out of publishing, it is aimed at public distribution, which serves a provider shipping an integration and does nothing for a team whose skills will never go to a market. Teams do group their own skills as a collection grows, but that is an organizational convenience rather than a unit of delivery. The protocol in this paper keeps the one portable unit, the skill, as the thing users act on, and leaves public bundle distribution to the existing plugin and marketplace path (Sec. E.8).

**Distribution.** The dominant registry, `skills.sh` [1], indexes public GitHub repositories and ranks skills by install count, and installation runs through a CLI (`npx skills add owner/repo`). A companion discovery convention lets any website self-host skills at `/.well-known/agent-skills/index.json` with content digests [34], though it is rarely used in practice, with Stripe the most prominent adopter. Publishing a skill therefore means, almost without exception, maintaining a GitHub repository. That keeps authorship technical even though the artifact is plain Markdown anyone could write (Sec. E.8). Our July 2026 crawl resolves the indexed corpus to 56,804 `SKILL.md` directories across 1,133 public repositories. We use this directory count throughout as the size of the ecosystem. It is an upper bound on unique skills, because aggregator mirrors and per-agent duplicates are deduplicated only in the full-corpus analysis (Appendix B).

**The competing convention: AGENTS.md.** In parallel, a much cruder mechanism has become the de facto standard for project-level agent instructions: a single `AGENTS.md` (or `CLAUDE.md`) file at the repository root, loaded in full on every message [35]. It offers no modularity and no sharing beyond the repository, and it charges full token cost for every instruction on every message. Yet by widespread practitioner report it is the most widely adopted way to deliver agent knowledge. What fills it is the telling part. These files carry the project’s and the team’s *workflows*: deploy procedures, review checklists, testing conventions, exactly the procedural knowledge skills were designed to package for a team.

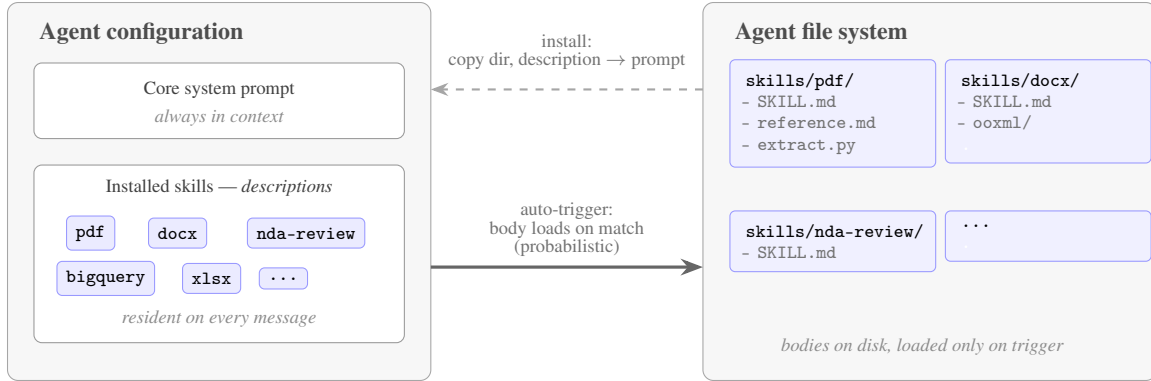


Figure 4: How installed skills work today (simplified from 3). Installation copies the skill directory onto the agent’s file system and places its one-line description (50–280 tokens in our measured setup) permanently in the system prompt. The body—SKILL.md plus scripts and references—stays on disk and loads only when the model’s probabilistic matching decides the description is relevant. Every installed skill pays the resident description cost on every message; whether it ever fires is left to the trigger lottery (Appendix C).

The role skills were built to play, a monolithic file plays instead. Appendix C explains why, and Appendix G returns to what its dominance implies.

## B The Skills Landscape: What Skills Have Become

Before analyzing how skills are *delivered*, we examine what the ecosystem actually *publishes*. Our July 2026 crawl resolves 56,804 SKILL.md directories across 1,133 public repositories; 56,245 skill bodies (99%) from 1,132 repositories were fully fetched and analyzed. Four findings shape the rest of the paper. Two measurement regimes run throughout, and we mark which is which at every figure: fetch-based measurements (counts, body sizes, repository concentration) cover the full corpus, while classification figures (category shares, external-service requirements) come from a 100-skill uniform sample carrying roughly  $\pm 10$  percentage points at 95% confidence. The structural findings below rest on the full-corpus measurements; the sampled classification corroborates them rather than carrying them.

**Finding 1: Skills are becoming a primary provider-to-agent integration channel.** The clearest structural signal in the corpus is who is publishing. Beyond individual authors and aggregators (repositories that re-host large collections of others’ skills), *first-party service providers* now maintain official skills repositories: AWS (aws/agent-toolkit-for-aws, 138 skills), Google (google/skills, 92), Elastic (elastic/agent-skills, 70), Grafana (49), LaunchDarkly (49), Sentry (80 across three repositories), Hugging Face (26), Expo (23), HashiCorp (17), Cloudflare (13, including a skill named simply wrangler), and Stripe, alongside Anthropic’s own official plugin collections (379). In total we identify 21 major-provider organizations publishing 958 first-party skills, and a naming convention (<org>/agent-skills or <org>/skills) hardening into the de facto address for “how agents drive our product.” The content confirms these are integration artifacts rather than prose. Of the 569 skills in the fifteen provider *product* repositories, 79% mention the provider’s own CLI or API tooling by name (aws, gcloud, wrangler, eas, terraform, sentry-cli, ...) and 60% contain executable shell blocks.<sup>2</sup> The broader classified sample agrees: 42% of skills require an account with an external service, and a further 26% wrap a free tool. This is the corpus-level view of the provider migration described in Sec. 1. Instead of maintaining an MCP server per agent, a provider ships one CLI plus one skill and reaches every agent that has a shell [28–32]. Two smaller specimens are equally telling. Stripe publishes the *same* six skills duplicated under per-agent directories (providers/claude/plugin/skills/, and so on), which is the per-vendor packaging fragmentation a shared delivery protocol removes. And its flagship skill’s one-line “description” runs to roughly 150 words of enumerated trigger conditions: the trigger-engineering pathology of Appendix C, practiced by a major provider.

**Finding 2: Skills are far too large to be resident, and just small enough to be fetched.** The median fetched skill body is 921 words and the 90th percentile is 2,207 (mean 1,159): on the order of one to four thousand tokens at the

<sup>2</sup>Method: per-organization term lists matched as substrings over skill bodies; shell blocks detected by fenced bash/sh code or npx/curl usage. These detect mention rather than verified execution and carry some false positives; word-boundary matching and hand validation would tighten the estimate, and we report it as an upper bound.



median, depending on tokenizer.<sup>3</sup> Keeping even a modest working set of skill *bodies* resident is out of the question. That is exactly why install-only delivery falls back to keeping only *descriptions* resident and gambling on auto-triggering (Appendix C). The same numbers cut the other way, though: a few thousand tokens fetched once, at the point of use, is cheap. The corpus itself argues for on-demand delivery.

**Finding 3: The integration layer is fragmented per agent—54 directories for one format.** The SKILL.md *format* is universal; where skills *live* is anything but. The community skills CLI keeps an agent registry, the code that must know every agent’s install location in order to work at all. It supports 75 coding agents using **54 distinct project-level skills directories**. Nineteen of the 75 have converged on `.agents/skills/`; the remaining 56 spread across 53 vendor-specific dotdirs such as `.claude/skills/`, `.windsurf/skills/`, `.devin/skills/`, and `.goose/skills/`, including, we must admit, our own `.adal/skills/` [1]. User-level storage is worse, at 58 distinct home-directory locations across the same agents. The CLI copes the only way it can, installing once and symlinking into every detected agent’s directory. The fragmentation is pure accident: every one of these directories holds the *same* file format and differs only in whose namespace it sits in. A team running two agents maintains two skill installations; an author documents  $N$  install paths; a repository accumulates one dotdir per agent its contributors run. Appendix D returns to this layer. The short version is that @skills unifies it mostly by removing storage rather than by standardizing it.

**Finding 4: The long tail already lives in registries, not in installs.** Publication is heavily concentrated: the two largest aggregator registries alone account for 11,894 of the 56,245 fetched skills (21%), and the top fifteen repositories for 51%. The ecosystem has already solved *hosting* and *indexing*. Tens of thousands of skills sit one HTTP request away, while delivery stays gated on per-user installation. The distribution infrastructure exists; the delivery protocol is the missing piece.

**What a full-corpus classification would add.** The four findings above are what the corpus establishes. A classification pass over the entire 56,804-skill inventory, rather than the 100-skill sample, would sharpen five quantities without disturbing them: (i) the share and growth of provider-official skills, and how many wrap a CLI rather than a per-agent integration; (ii) the prevalence and token cost of trigger-engineered descriptions (descriptions padded with invocation phrases to win the auto-trigger lottery); (iii) exact token-cost distributions of descriptions and bodies under a production tokenizer; (iv) the category structure of what the world publishes, where our sample gives 30% software engineering, 25% creative production, 16% writing, and 15% design, with a long tail of business functions; and (v) duplication and staleness across aggregators. We flag these as refinements of magnitude rather than open questions of direction: each measures how strongly the ecosystem’s binding need is a delivery protocol rather than more install tooling, and none of the paper’s claims turn on where in the confidence interval they land.

**The landscape, distilled into a definition.** The official definition (Appendix A) says what a skill *is*. The corpus shows what skills are *for*. We therefore extend it:

*A skill is procedural knowledge for an AI agent, either the world’s expertise or a team’s own way of doing things, and it serves three roles: domain knowledge, expertise distilled into instructions; provider integration, by which a service teaches any agent to drive its product; and team workflows, the customized procedures a team adapts to its own way of working. A model knows what the world knows; a skill adds what it doesn’t.*

The rest of the paper is about giving each of these three roles the delivery it needs.

## C Why Install-Only Delivery Fails

The failure is architectural: install-only delivery makes prompt residency plus probabilistic matching the *only persistent* channel through which any of 56,804 published skills can reach an agent. The non-installing path now offered alongside it (Sec. 1) delivers a single named skill per invocation and caches nothing, so it relieves none of what follows: anything that must be there next session, anything at collection granularity, and anything whose cost is paid per message still goes through installation. Nothing in this section indicts auto-triggering itself; implicit activation is genuinely valuable, and the tier model retains it for exactly the instructions that need it (Appendix D). Installed skills can also be invoked by name in the prompt, but that deterministic path exists only for the handful already installed and already paying residency; install-only delivery offers no explicit path to the rest of the corpus. This section measures the channel at three levels: it is physically scarce; authors and users rationally route around the scarcity; and the ecosystem inherits the damage. Six mechanisms, three levels, one cause.

<sup>3</sup>At 1.3 to 4 tokens per whitespace-delimited word, spanning typical prose ratios to a code-heavy upper bound, the 921-word median body is roughly 1.2k–3.7k tokens and the 90th percentile 2.9k–8.8k. We report the range rather than a point estimate because the exact figure is tokenizer-dependent; the argument here needs only the order of magnitude, which holds across the range.



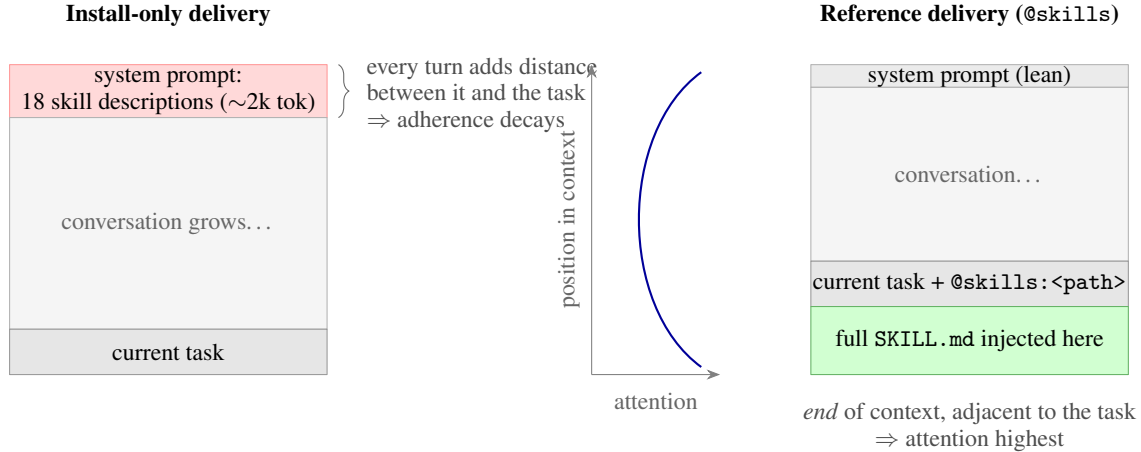


Figure 5: Delivery position is the mechanism. Left: an installed skill’s description stays fixed at the top of the context, so the growing session pushes the task ever further from it, and adherence to it decays with the distance [6, 7]. Right: an explicit reference injects the full skill at the end of the context, adjacent to the task—the mechanically most reliable position an instruction can occupy. Center: attention as a function of position (schematic).

### C.1 Level 1: The channel is physically scarce

Three independent mechanical properties of resident context, each documented in the literature, jointly cap what the channel can carry.

**Distance decay: the request drifts away from the instruction.** A resident description sits at a fixed place, the top of the context, and it stays there. What changes is the distance to the request it is supposed to match: every turn of conversation, every file read, and every tool result lands between the two, so an instruction that started next to the first request is separated from the tenth by everything that happened in between. Transformer language models attend most strongly to the beginning and the end of their context and under-weight what lies between [6, 36]. The bias is intrinsic and U-shaped rather than semantic, present regardless of how relevant the content is [37], and prompt-level fixes help only inconsistently [36]. Two lines of evidence measure the effect on instructions directly: adherence to the system message decays over conversation turns [7], and models in long multi-turn settings lose an average of 39% against single-turn performance on the same tasks [8]. The failure is positional, not semantic. The same instruction that goes unheeded from the top of a long session is followed when it arrives at the end of the context, next to the task (Fig. 5). Explicit reference (Appendix D) exploits exactly that.

**The standing tax: every install taxes every message.** Each installed skill’s description stays resident permanently, paid on every message whether or not the skill is relevant. Measuring the installed skills on one real working setup, we find 50–280 tokens per skill; that session, with 18 installed skills, carries roughly 1,500–2,000 tokens of standing overhead on every request. The cost is not only financial. Input length alone degrades reasoning on an otherwise unchanged task, well below the model’s nominal context limit [9]; effective context is smaller than advertised context across model families [10]; and prompt bloat is a recognized enough problem that an entire line of work exists to compress prompts back down [38].

**Dilution: each additional description weakens the rest.** More resident descriptions means more competition for the model’s attention, and each individual skill fires *less* reliably. Irrelevant context is not neutral—models are measurably distracted by it even when instructed to ignore it [11]—and instruction-following degrades systematically with the number of simultaneous instructions: at 500 concurrent constraints even the best frontier model reaches only 68% adherence, with degradation visible from far smaller counts [12]. The mechanism degrades exactly as adoption succeeds: the better the ecosystem does at getting skills installed, the worse each installed skill works. Bundling amplifies the problem, because the unit of installation is coarser than the unit of cost. Plugins are frequently loosely curated, and a single plugin can carry twenty or more skills; the dominant harness offers no per-skill control inside a plugin—install, enable, and disable all operate on the whole bundle [13]. A user with a handful of plugins is quickly past a hundred resident descriptions, more entries than the agent’s own built-in tool set, each one diluting the rest.

**The capacity: an attention budget of fewer than a hundred slots.** Jointly, the three mechanics cap what the channel can carry. We do not claim a precise constant; no controlled measurement of trigger reliability versus installed-skill

count yet exists, and we propose one as future work (Sec. 4). But the evidence bounds the budget well under a hundred. Instruction-following degrades measurably as concurrent instructions accumulate, long before the hundreds [12]; that experiment measures concurrent constraints rather than trigger descriptions, so we take it as suggestive rather than direct. Practitioner guidance on resident context consistently counsels parsimony [39]. We therefore claim only a conservative bound: *fewer than one hundred reliable auto-trigger slots per agent*, and plausibly far fewer. The framing matters more than the digit: installation is an *attention budget*, not a distribution channel—a physical property of resident context, not anyone’s design mistake. Spending a slot is justified for an instruction that must fire unprompted; the pathology is that install-only delivery forces *every* skill, however occasional its use, to bid for one. And wherever the true budget lies below one hundred, it is orders of magnitude smaller than 56,804.

## C.2 Level 2: Everyone rationally routes around the scarcity

Given a scarce, probabilistic channel, every party adapts—and each adaptation makes the channel worse.

**Authors: trigger-engineering.** An installed skill fires only if the model matches the user’s message against its one-line description; matching competes with everything else in context, and misses are silent—the user never learns the skill existed. Authors respond rationally: examine popular published skills and the descriptions are dominated not by documentation but by *trigger-engineering*:

```
TRIGGER -- read BEFORE opening the target file; don't skip because it "looks like a
one-liner"-- whenever: the prompt names [...] in any form [...] SKIP only when
[...]
```

followed by paragraphs of trigger and skip conditions. This is a rational bid for a scarce slot, and it backfires collectively: among the examples we examined, TRIGGER-block descriptions cost roughly 20× more tokens than quiet ones, feeding the tax and the dilution of Sec. C.1 (full-corpus prevalence is part of the pending analysis, Appendix B). Even major providers play this game: Stripe’s flagship skill carries roughly 150 words of enumerated trigger conditions. Such blocks among widely-installed skills show authors pricing in, correctly, that the channel is unreliable.

**Users: installing is the wrong unit for trying something out.** For a user, the cost of an install is certain and paid on every message (Sec. C.1), while the benefit is occasional and probabilistic. Installing is therefore worth it only for a skill that is used often, that clearly benefits from firing without being asked, and that is known to trigger reliably. Everything else falls outside those bars: skills being tried out, skills being compared against alternatives, skills needed once for an unusual task. For all of these, install-only delivery charges the full price of permanence. The user must place the skill, carry its resident description on every message thereafter, judge whether it is pulling its weight, and later find and remove it, in each agent and on each machine. That is heavy management for something the user wanted to *try*, and the natural response is to not bother, which is how most of the corpus goes unexplored. Users do keep installing past that point, and the install itself is the easy part. What breaks down afterwards is memory: because the skill lives in a hidden per-agent directory and is meant to fire on its own, the user stops holding onto what is installed. When it then fails to fire (Level 1), nothing prompts them. They do not know a skill that would have helped was sitting right there, so they cannot invoke it by name either. Installation quietly transfers responsibility for remembering to a mechanism that only sometimes remembers. How many installs users actually settle at, and how trigger reliability falls as that number rises, are open empirical questions we propose measuring (Sec. 4).

**Teams: defection to AGENTS.md.** Teams needing shared procedures skip the channel entirely: they paste workflows into a monolithic AGENTS.md (Appendix A)—always loaded, zero setup, no trigger lottery. The crudest delivery mechanism wins because it beats installation on the only axis users feel: simplicity. But the defection buys no reliability: AGENTS.md is resident context too, subject to exactly the distance decay of Sec. C.1, hence the ubiquitous complaint that the agent “ignores” the team’s instructed workflows, especially as the conversation grows long. The irony is sharp: a team’s workflows are typically a manageable number—a few dozen at most—precisely the working set that skills were designed to carry, packaged one procedure per file. Teams thus end up, for lack of an alternative, on a mechanism that fails the same way. Adoption behavior has gone decisively against the install channel; Appendix G returns to what that implies.

## C.3 Level 3: The ecosystem inherits the damage

**The junk drawer: important skills get lost among clutter.** Because install is the *only* lifecycle, everything lands in the same flat bin: a skill grabbed once for a one-off task sits forever next to the playbooks a team uses daily, and nothing distinguishes them. Users create skills expecting auto-trigger to remember for them, we conjecture, and so stop remembering themselves; then auto-trigger does not fire (Level 1), and the skill is simply forgotten. The consistently

used, genuinely important procedures have no dedicated home and no management surface; they rot in a pile of once-used clutter. The resulting experience (install, forget, stop trusting, stop installing) is plausibly among the larger brakes on adoption of the entire ecosystem: a newcomer’s first contact with skills is a directory of things that never fire, from which they conclude that skills do not work.

**The cap: distribution ends at each user’s top handful.** The structural consequence of all six mechanisms:

*56,804 indexed skills compete for fewer than 100 reliable auto-trigger slots per agent.*

The long tail has no delivery path; authors face zero measurable usage and stop publishing; users conclude the ecosystem is thin and stop looking. The install lifecycle—not the file format—is what strangles the ecosystem.

## D The Three-Tier Delivery Model

The failures of Appendix C share one root. Installation is the only tier, so every skill pays the same maximal cost, permanent prompt residency, for the same scarce benefit, a chance at auto-triggering, whether it is used once, weekly, or on every message. The fix is to let each skill sit in the *cheapest tier that meets its activation need*. Splitting content, persistence, and triggering apart (Sec. 1) yields exactly three tiers (Fig. 1). The three roles of Appendix B map onto them directly: domain knowledge is the tier-1 long tail, team workflows are the tier-2 working set, and provider integrations reach agents through tier 1, plus tier 3 for the few that must fire unprompted.

**Tier 1: Reference.** @skills:<path> fetches a skill at the moment of use and injects its full body at the end of the context, next to the task, which is where attention is highest (Sec. C.1). Triggering is explicit and therefore deterministic, the standing cost is zero, and the skill evaporates with the session. This tier serves the entire long tail: under reference delivery every published skill has a real chance of being used, because none of them has to win an install slot first.

**Tier 2: Saved.** A saved skill loads exactly like a reference, on demand and triggered explicitly, but it stays. It lives in a git-tracked directory in the repository, .atskills/ in our implementation, where the team can find it, edit it, and version it alongside the code. Crucially, *nothing* enters the prompt. The agent indexes .atskills/ for @ autocomplete and search, so saved skills surface the moment the user types, but that index lives in the tooling, outside the model’s context. Zero resident tokens, zero trigger dilution, zero per-machine setup. This tier keeps the good half of installation, persistence and findability, and drops the bad half. It is per project rather than per machine, it arrives with git clone rather than through a teammate’s install, it works offline, and resolution is instant because the files are already local. Tier 2 is the most underserved need in the current ecosystem, and it is where a team’s *working set* belongs: the deploy runbook, the review checklist, the migration procedure. It is also the better home for a fat AGENTS.md, the same instructions split into one procedure per skill, each loaded when invoked instead of taxing every message.

**Tier 3: Installed.** Installation stays the right mechanism for one class of skill: the ones that must fire *without the user thinking of them*, such as formatter conventions, security guardrails, and API-reference checks. Even here, only the frontmatter, the name and one-line description, is resident; the body loads on trigger, exactly as the format prescribes (Appendix A). Because auto-triggering is a bounded attention budget (Sec. C.1), tier 3 should stay small, and we recommend fewer than ten essentials, since every addition dilutes the rest. The protocol does not replace installation. It relieves it of the tens of thousands of skills that never belonged in the system prompt.

**Unified storage.** The three tiers map onto a unified storage story (Fig. 3), reusing existing conventions where they exist and introducing only what is missing:

- **Tier 1** materializes nothing, or—for skills that ship scripts—writes into a session-scoped temporary directory, deleted after the session.
- **Tier 2** introduces one new location: .atskills/ at the repository root, the persistent home of referenced skills a team decided to keep. As of July 2026, no agent claims this namespace. Saves are *vendored* at the ID’s own path—hub skills under their publisher’s namespace (.atskills/<owner>/<skill>/), GitHub skills under .atskills/gh/<owner>/<repo>/...—so skills from one source nest together, the tree stays readable at any size, and every saved copy answers its own address (Sec. E.2).
- **Tier 3** needs no new storage at all: install is *one line* in .atskills/.autotrigger, a file with .gitignore semantics (Sec. E.4). A plain line auto-triggers a skill the project holds; an @ line *follows* a provider’s skill without holding a copy; a trailing / takes a whole directory. The frontmatter stays the author’s and activation becomes the consumer’s, and one readable file shows the project’s whole auto-trigger budget as one-line diffs any reviewer understands. Provenance needs no manifest either: a saved skill carries a two-line .source stamp, and a followed skill’s provenance *is* its line. Installation under the protocol is therefore project-scoped

and git-tracked. The protocol defines no user-level install of its own and offers the hub instead (Sec. E.8). Package management settled this scope question long ago: global installs lost to per-project dependencies, `npm -g` to `node_modules` and system `pip` to virtual environments, because global state is invisible, forgotten, and unreproducible. A hidden home directory is where installs go to be forgotten, every agent renders it through its own interface, and the symlink bridge copies the same skill across 58 home-directory locations (Appendix B). Existing vendor directories such as `.claude/skills/` and `.agents/skills/` stay readable for compatibility—the protocol feeds agents that load skills their own way; it does not replace them (Sec. E.6).

**User-level skills: the hub, not a dotdir.** One thing is deliberately absent: any *new* user-level directory for the working set. We considered one and rejected it. A hidden dotdir outside the working tree is where skills go to be forgotten, since nobody revisits it and no editor or code review surfaces it, and it is awkward for developers and unreachable for everyone else. Personal and cross-project collections live in the hub instead (Sec. E.8), managed in a dashboard rather than a dotdir. They lose nothing in immediacy, because the agent surfaces them directly: the @skills autocomplete lists the user’s cloud collection next to the project’s saved skills, in any project and on any machine. The user gets the availability a home directory promises with the visibility it never delivers. The one thing personal skills give up is implicit activation, and we state that as a rule rather than a loss: *auto-triggering is a project decision*. A personal skill that should fire automatically in some repository gets installed into that repository, one act and git-tracked (`:install`); everywhere else it stays one @ away. Existing home-directory skills keep loading exactly as before. The protocol simply defines no user-level store of its own, and the hub is the alternative it offers, not a migration it forces.

**A lifecycle, not just a cost model.** The tier model is also a lifecycle, and that is what fixes the junk drawer (Sec. C.3). One-off skills never persist at all, because tier 1 evaporates with the session. Persistence happens only through *deliberate promotion*: saving a referenced skill into `.atskills/` is an explicit act (Appendix E), so the saved tree holds exactly what someone decided to keep. It is visible in the repository, reviewed like code, and maintained by the team, instead of an invisible per-machine directory collecting everything ever tried. Promotion is also the moment of *adaptation*, since a saved copy can drift from its upstream source as the team fits it to their project. Through promotion and local edits, a generic public skill becomes the team’s own practice.

The tiers complement each other rather than compete. A marketplace can still emit native install commands for the handful of essentials a user wants auto-triggered, while tiers 1 and 2 serve everything else ever published. Nothing asks the user to change: personal skills, plugin installs, and vendor directories keep loading as before, a project-saved copy deterministically shadows a same-name duplicate, and installing becomes one tier of three. The protocol does not replace the existing system. It *completes* it.

The tier model is also where this design meets emerging industry guidance on context engineering: treat context as a finite resource with diminishing returns, and prefer just-in-time retrieval of lightweight identifiers over pre-loading content [39]. The three tiers put that guidance into practice for skills, and go one step further. Tier 2’s search index sits *outside the context altogether*, in the tooling, so even the lightweight identifier costs nothing until the user types @. Tiers 1 and 2 *are* just-in-time retrieval, with the piece the guidance lacks: an ecosystem-wide protocol and catalog behind the fetch (Appendix E).

**What the user gains.** The benefits compound into a different experience of skills:

- **Granularity.** Save or install *one* skill instead of a twenty-skill plugin, so the unit of control finally matches the unit of cost (Sec. C.1).
- **Transparency.** Every skill has exactly one knowable place: the project directory, a git diff away; the hub, a page with an owner and a history; or the session transcript. The project’s whole auto-trigger budget is one readable file.
- **Clean prompt, free management.** The system prompt carries only the few essentials someone deliberately chose, and everything else costs nothing until the moment of use. Prompt hygiene stops being a chore because there is nothing to clean.
- **Robust manual triggering.** An explicit reference loads every time, at the best position for compliance, with no lottery and no silent misses.
- **Sharing.** Git carries the project set to every teammate on clone, the hub carries personal and team sets to every machine, and publishing needs no repository.
- **Unification.** One syntax, one project directory, one trigger file, one hub, and the same experience in every agent at either integration level.

## E The @skills Protocol: Full Specification

The protocol is deliberately small enough to state completely. It consists of four file-level mechanisms—a reference command (@skills:<path>), a project folder (.atskills/), a trigger file (.autotrigger), and a provenance stamp (.source, the two lines recording a saved copy’s origin and the revision it was taken at)—plus a management surface (/skills) that every conforming client should ship. Throughout, the unit of content is the unmodified SKILL.md directory of Appendix A: the protocol adds *zero* fields to the format. It is developed in the open, with the specification, the agent instruction file (SKILLS.md), a reference CLI, and a runnable demo maintained at <https://github.com/SylphAI-Inc/atskills> [17]. AdaL implements the protocol end to end—native references, autocomplete, the global validating cache, saves, the auto-trigger file, and the /skills checkbox tree—in both its CLI and browser application [18]. Appendix F states the four normative algorithms; this section specifies the artifacts and their meaning.

A design invariant governs everything below: **the protocol is purely a filesystem**—folders, files, lines, nothing else. There is no manifest, no lockfile, no database, and no state that is not visible in an editor and reviewable in a pull request. A second invariant bounds the state space: there are exactly **two states**. *Theirs*: a skill followed by one line in .autotrigger, never held locally—the provider keeps it true because it documents the provider’s service. *Yours*: a folder in .atskills/, either written in place or saved-to-adapt, detached from upstream at the moment of saving. The muddy third state of every install system—installed-but-stale, enabled-but-unknown-origin, update-pending—is designed out, not managed.

### E.1 Reference syntax and identity

A skill is referenced inline, in the user’s message or by the agent itself. The *path is the identity*:

|                                 |                                           |
|---------------------------------|-------------------------------------------|
| @skills:hub:sylphai/glowmotion  | one skill, from the hub                   |
| @skills:gh:acme/skills/deploy   | one skill, straight from GitHub           |
| @skills:gh:stripe/agent-toolkit | a directory -> a menu, one line per skill |
| @skills:<path>:save             | copy into the project -- to adapt it      |
| @skills:<path>:install          | append its line to .autotrigger           |
| @skills:<path>:save:install     | both -- your copy, firing on its own      |

Identity rules: hub paths (hub:owner/name) are lowercase, and resolvers fold case so it can never split an address; gh:owner/repo/sub keeps GitHub’s casing beyond the marker, because GitHub paths are case-sensitive. Pasted GitHub URLs (github.com/.../tree/<branch>/...) normalize to the gh: form. On disk, gh: is spelled gh/, since folder names cannot hold colons. One skill has one ID, forever: GitHub-hosted content is always gh:, even when the hub indexes or serves it (Sec. E.8).

The grammar is greedy: the path runs until the end of the token or the trailing suffixes. The two suffixes are *orthogonal and combinable*—:save means own a copy, :install means one line in .autotrigger—and neither implies the other. Several references in one message all load, each at its own point of use, which is what makes multi-skill workflows deterministic: install-only delivery has no equivalent, since co-firing  $N$  skills means  $N$  descriptions winning a probabilistic match at once (Sec. C.1).

**A directory is a menu.** A path with no SKILL.md is not a failure—it is a directory, and it loads as a menu: one line per skill under it, path: description, every line itself a valid reference. Browsing and using are the same gesture, and a bundle is only ever a menu: skills are taken one path at a time, and all-or-nothing delivery does not exist in the protocol. The menu is produced by the *leaf rule*: a folder holding SKILL.md is a skill and the walk stops there, so nested repositories can never yield a skill inside a skill, and repository cruft (README.md, LICENSE, docs/) is never mistaken for content. Only a SKILL.md at the referenced path itself makes the reference a single skill.

**The autocomplete rule.** @skills: completes only what the project already knows: its local skills and the cloud IDs in .autotrigger. For the world, the user types or pastes a path—GitHub URLs work—and discovery is the hub’s job, not the input box’s. The learning curve is one character.

### E.2 Resolution: by path, through a validating cache

Resolution has one local rule and one cloud rule (Fig. 6, Algorithm 1):

1. **Local first, by path.** A folder at .atskills/<path> (gh: spelled gh/) is the project’s own and always answers; read it, use it, stop. No folder there means the path means the cloud. Nothing else is consulted—in particular, .source (Sec. E.3) is *never* read to resolve anything. Because a saved copy sits at its ID’s



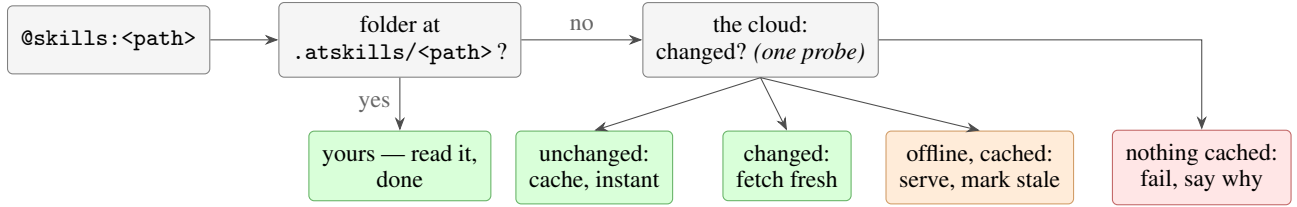


Figure 6: Resolution is by path, through a validating cache (Algorithm 1). A folder at `.atskills/<path>` is the project’s own and always answers—a saved copy sits at its ID’s own path, so it answers its own address with no lookup machinery. Otherwise the path means the cloud: one revision probe decides between serving the shared global cache instantly, fetching fresh, or (offline) serving the cache marked stale; every green outcome injects at the point of use. Oversized references—a whole marketplace repo as one path—are refused before any download, with the loadable sub-collections named.

own path, it answers its own address by construction: this is vendoring, with deep precedent—Go’s `vendor/github.com/acme/...` mirrors the import path and resolves first, node’s `node_modules/@scope/pkg` keeps the scope. Move or rename the folder and it simply stops answering the old address.

2. **Else the cloud, through the global validating cache.** Cloud content materializes under one machine-wide, agent-neutral cache root (`$XDG_CACHE_HOME/atskills/<disk path>`, defaulting to `~/.cache/atskills/<disk path>`), shared by every conforming client on the machine. This root **must not** live inside any `.atskills/` directory: a project whose root is the home directory would otherwise enumerate the machine’s cache as its own skills, and an auto-trigger line written against a cached copy is git-tracked but machine-local, resolving to nothing on a teammate’s checkout. The cache has browser semantics: each use asks the source “did this change?”—a single revision probe, never a re-download—and *unchanged* serves the cache instantly, *changed* fetches fresh, *unreachable* serves the cached copy with a stale warning, and *unreachable-with-nothing-cached* fails and says exactly why. Entries are always safe to delete; the path re-resolves.

The reference transport for GitHub content is git itself: one shallow, blob-filtered, sparse clone of the referenced sub-path, and one `ls-remote` for the change probe. This choice is deliberate. Git negotiates the transfer in one round trip, needs no API quota (REST-based transports die at unauthenticated rate limits in practice), works against private repositories through the user’s existing credentials, and gives revisions for free: the probe is a commit hash, and a pinned-revision fetch (needed by save-again verification, Sec. E.3) is a plain fetch-by-sha. A machine without git gets one clear error naming the missing tool—not a slower hand-rolled fallback.

### E.3 `.atskills/` and `.source`: save = adapt + detach

Everything under `.atskills/` is the project’s: either written in place (any name, no stamp), or saved because someone intends to *adapt* it. Saving is a download, not a subscription—the copy detaches from its provider the moment it lands. A provider skill nobody will edit does not belong here at all; it is followed with one line (Sec. E.4) and never held.

```

.atskills/
|-- .autotrigger          what fires on its own
|-- my-tdd/SKILL.md       yours -- any name, no .source
|-- hub/sylphai/glowmotion/ saved to adapt -- the path is the ID
|   |-- SKILL.md
|   |-- .source
|-- gh/stripe/agent-toolkit/ saved directory -- one .source covers it all
|   |-- .source
|   |-- payments/SKILL.md
|   |-- terminal/SKILL.md

```

`.source` is two lines, written once at save time and never touched again:

```

gh:stripe/agent-toolkit
2026-08-01 rev:abc123...

```

Line 1 is the origin ID; line 2 is the date and the upstream revision taken. It is pure provenance—a birth certificate, not a leash: the resolver never reads it, nothing syncs against it, and deleting it detaches fully. A subtree save writes *one*

stamp at the top of what was saved; the closest `.source` at or above a skill is its origin. Absence of a stamp is itself information: no `.source` means the project wrote it, and the cloud is never consulted about it.

**No update lifecycle—save-again instead.** A saved skill is detached on purpose, so there is nothing to “update”: edit it, commit it, git carries the history. Curiosity about upstream is a question, not a lifecycle: the agent reads `.source`, fetches, and diffs, with line 2 separating “you changed it” from “they changed it.” Saving the same path again is the only refresh, and it is conflict-safe by construction (Algorithm 2): an *unedited* copy is replaced and line 2 rewritten, where unedited is verified by re-fetching upstream *at the recorded revision* (immutable, fetchable by hash) and comparing bytes—no content digests, no staging directories, no stored state beyond the two lines. An *edited* (or unverifiable) copy is a conflict, and a conflict touches nothing and lists the ways out: keep yours (do nothing); refetch (delete the folder and save again—git keeps the history); or merge (ask the agent, with line 2 as the base). There is deliberately no three-way text merge in the protocol: reconciling two versions of prose instructions is a semantic judgment, and the judge is already in the terminal. Remove is deleting the folder. No other machinery exists.

**No pinning, ever.** The protocol defines no versions, releases, or lockfiles. A skill documents a living service; freezing the text while the API moves is false safety—old instructions for a world that did not freeze. The two honest relationships are *follow* (the provider keeps it true) and *own* (you hold the text and own keeping it true); real control over what the agent reads is a local copy, and that is exactly what save is for.

#### E.4 `.autotrigger`: install is a line

One file governs everything that fires on its own, and it works like `.gitignore`: one entry per line, `#` for comments, duplicates load once, order preserved.

```
# .atskills/.autotrigger
sec-checklist           yours -- lives in .atskills/, reviewed
team-flows/             yours -- every skill under the directory
@hub:sylphai/glowmotion hub -- follows the author's latest
@gh:stripe/agent-toolkit/payments github -- follows Stripe's latest
@gh:stripe/agent-toolkit/ github directory -- all of it
```

The line tells you everything: plain = your file, `@` = the cloud, `@gh` = GitHub, trailing `/` = the whole directory. A plain line is a gitignore *pattern* over the local skill tree—globs and `!` negation included, with all plain lines forming one ruleset so negation composes exactly as it does in git. **Install = a line in this file.** That is the whole equation: installing a skill is adding its line, uninstalling is removing it, and the suffixes, the `/skills` checkboxes, and hand edits in any editor all write the same lines.

What “fires on its own” means, precisely (Algorithm 3): at session start each resident skill contributes its *frontmatter only* (name + description, ~50–100 tokens) to the always-available index; the body loads only on trigger—the format’s own progressive disclosure. Cloud lines refresh once per session through the validating cache, so an unchanged upstream costs one revision probe; offline serves the last cached copy, marked stale. Every line resolves local-first, so a saved copy answers its own `@` line and what fires is the team’s adaptation. Per-line failures are isolated: a line that loads nothing is reported once and the session goes on. A directory line’s cost is its children’s frontmatter, and the management surface always shows the expanded count and cost, never one opaque line; a directory line’s drift includes entirely new skills arriving under it, and the count change is shown at session start.

**The trust trade, stated honestly.** An `@` line is reviewed once, as a one-line diff, and then the provider’s updates flow in unreviewed—that is what following *means*, and the management surface marks it. Scripts are confirmed *by change, not by location*: the first run of a cloud skill’s script shows the command and asks; it asks again only when the skill’s revision changed since the last confirmed run (the cache already knows the revision). A followed provider skill therefore never nags, and saving is never needed just to silence confirmations. Follow what’s theirs; save what you’ll make yours.

#### E.5 `/skills`: the management surface

Part of the protocol, not the product: non-technical users are first-class, so every conforming client should ship the management surface, and the honest claim is “no state beyond the files,” not “no management surface.” `/skills` is a view and editor over `.atskills/` and `.autotrigger`—nothing else:

```
/skills
[x] team-flows/           check a whole directory
[#] deploy
[#] review-checklist
```

```
[ ] my-tdd
[x] @gh:stripe/agent-toolkit/payments    cloud - auto-updates
                                         budget: 4 skills - ~260 tokens
```

Check a box and a line is written; uncheck and it is removed. The checkbox states mirror the file faithfully: checked directly ([x], its own line), covered by a directory line or pattern ([#]), partially covered ([~]). Unchecking one skill under a covering directory line performs a *split*: the directory line is replaced by explicit lines for the siblings that stay on, so the file always reads true (Algorithm 4). Every operation is equally a typed command (/skills save | install | uninstall | remove | toggle <path>), and the surface shows each skill’s origin (*yours / saved from... / cloud*) and the expanded cost of every line. The key affordance is **view prompt**: the exact text injected into the model, word for word, with its token count. You read what the model reads; no agent offers this today.

## E.6 Adoption Level 0: one file or one CLI, zero integration

The protocol asks nothing of the agent or its vendor—no SDK, no registry client, no plugin. SKILLS.md, one instruction file shipped in the protocol repository and droppable into any repository, turns any agent that can read files, run shell commands, and fetch URLs into a full client: it states the resolution rule, the cache discipline, the .autotrigger semantics, the save procedure, and the safety rules, in about a page. The agent *is* the integration.

The same repository ships the second zero-integration path: a small dependency-free reference CLI. An agent that shells out to it inherits the cache, the resolution order, and the save rules without implementing any of them:

|                      |                                                   |
|----------------------|---------------------------------------------------|
| atskills get <path>  | resolve local -> cache -> web; print the files    |
| atskills save <path> | copy to .atskills/<path>/ + .source               |
| atskills triggers    | parse .autotrigger; print what loads and its cost |
| atskills prompt      | the exact resident block, word for word           |
| atskills skills      | the /skills management surface                    |

Either way, a resolved skill is just files on disk. Agents that load skills their own way—skill flags, native folders—keep working untouched: the protocol feeds them; it does not replace them.

## E.7 Adoption Level 1: native @ integration

Level 1 lives in the agent, and it is deliberately small because it reuses the @ context system every modern agent already has: references are detected exactly where @file mentions already are, resolve by Algorithm 1, and stream content into the context at the point of use; autocomplete surfaces the project’s local skills and its followed cloud IDs in the same dropdown users already use for files.

One architectural property makes Level 1 cheap and is worth stating as part of the protocol: **the host needs no protocol logic**. In AdaL’s implementation, the frontend client owns everything—parsing .autotrigger, resolution, the validating cache, saves, and the construction of the resident prompt block—and the model-serving backend receives that block as *one string* it splices into the system prompt verbatim. The protocol implementation therefore lives in exactly one codebase per client, the same one that renders the management surface, and the resolution the user sees in the dialog can never disagree with what the model receives. Any agent architecture with a “prompt assembly” step can adopt the protocol this way, whatever language its serving layer is written in.

## E.8 The hub: finding and authoring as a free service

A protocol alone does not solve finding, so it pairs with a free hub (<https://atskills.one>) [40]. The hub is a service, never a requirement—gh: and local paths resolve with zero hub involvement, and hub paths are plain HTTP GETs anyone can mirror. The governing precedent is git and GitHub: git is fully open, nobody is forced onto GitHub, and GitHub became indispensable by being the best place rather than the required place. Forcing the hub would kill both sides—a protocol that funnels users to one server is not a protocol, and an unadopted protocol brings the hub no crowd—so gh: stays first-class forever, and the hub competes only on the jobs GitHub is genuinely bad at: search over the 56k-skill corpus, one-screen authoring for the non-developers whose procedural knowledge skills capture best, private and team collections, and the user’s own library.

Identity honesty is a hard rule: for GitHub-hosted skills the hub is a mirror plus search with enriched metadata—categorization, better descriptions, rankings—and nothing more; identity stays gh: even when the hub indexes, serves, or features the skill. Only skills authored on the hub carry the hub’s own identity. The hub never renames other people’s work, so the aliasing problem cannot exist. For curation the hub uses *playlists*: named lists of gh: IDs, searchable

and one @ away, behaving exactly like directories everywhere else in the protocol—@-able as a menu, savable as a set, installable as one line—the way a music service links tracks it does not own.

**Bundles: granular by design.** The protocol deliberately defines no bundle-level delivery: the unit of delivery is a single skill because the unit of cost is a single skill (Sec. C.1), and a directory or playlist is always a menu, never an all-or-nothing install. Plugin and marketplace installs are not removed; they keep working unchanged alongside the protocol for users who want bundles. Scope is bounded the same way: the protocol distributes *documentation*. Plugin machinery that installs capability—hooks, MCP servers, slash commands—is a different problem and stays out; what dissolves here is the documentation half of “plugins,” which is most of it.

## F The Protocol as Algorithms

The protocol is small enough to state completely. This appendix gives the four normative algorithms—resolution, save, residency, and the checkbox toggle—exactly as the reference implementations execute them [17, 18]. Two properties are worth reading them for. First, *statelessness*: every algorithm is a function of the filesystem (the project tree, the trigger file, the stamps, the cache) and the upstream; there is no database, no manifest, and no state that a user cannot inspect in an editor. Second, *honest failure*: every refusal names its reason and, where one exists, the reference that would have worked.

Notation:  $K = 128$  is the collection cap;  $\text{disk}(id)$  spells gh: as gh/;  $\text{walk}(d)$  enumerates skill directories under  $d$  by the *leaf rule* (a folder holding SKILL.md is a skill and the walk stops there, so a nested SKILL.md belongs to the bundle above it);  $\perp$  denotes “unreachable.”

### F.1 Resolution

---

**Algorithm 1** RESOLVE( $p$ ) — local first, by path; then the validating cache

---

```

1:  $id \leftarrow \text{NORMALIZE}(p)$                                 ▷ fold hub case; gh: keeps casing; GitHub URLs  $\rightarrow$  gh:
2:  $d \leftarrow \text{.atskills}/\|\text{disk}(id)$ 
3: if  $d$  is a directory with content then                    ▷ local first — .source is never consulted
4:   return DESCRIBE( $id, d$ )
5: end if
6: return READTHROUGHCACHE( $id$ )

7: function DESCRIBE( $id, d$ )                                ▷ a skill, or a directory-as-menu
8:   if  $d/\text{SKILL}.md$  exists then return skill: body + bundled files
9:   end if
10:  require  $|\text{walk}(d)| \leq K$                                 ▷ else refuse, listing the largest loadable sub-collections
11:   $M \leftarrow \{ (id/rel, \text{frontmatter}) : rel \in \text{walk}(d) \}$ 
12:  if  $M = \emptyset$  then fail “nothing at  $id$ ”
13:  end if
14:  return menu  $M$                                           ▷ one line per skill; every line a valid path
15: end function

16: function READTHROUGHCACHE( $id$ )                          ▷ browser semantics over git
17:   $c \leftarrow \text{cache}/\|\text{disk}(id)$ ;  $m \leftarrow \text{meta}(id)$ 
18:   $r \leftarrow \text{HEADREVISION}(id)$                           ▷ one git ls-remote — a probe, never a download
19:  if  $c$  cached and  $r \neq \perp$  and  $r = m.\text{rev}$  then
20:    return DESCRIBE( $id, c$ )                                ▷ unchanged: served instantly
21:  else if  $c$  cached and  $r = \perp$  then
22:    return DESCRIBE( $id, c$ ) with stale warning              ▷ offline: cache, marked
23:  end if
24:  try FETCH( $id, c, r$ );  $\text{meta}(id) \leftarrow (r, \text{now})$ ; return DESCRIBE( $id, c$ )
25:  on failure: serve the cached copy with a stale warning if one exists; else fail with the reason
26: end function

```

---

FETCH for gh: content is git in three phases, and the order is the security and scale property. *Phase 1, trees only*: a shallow, blob-filtered, no-checkout clone (`-depth 1 -filter=blob:none -no-checkout, -sparse` when a sub-path is given) transfers the directory structure but not one file body. *Phase 2, count before paying*: `git ls-tree`

-r -l lists every path with its blob size from tree metadata alone; the leaf-rule count over the referenced sub-path enforces  $K$  *before* any content downloads, and a refusal reports the reference’s true weight (“6,296 skills, 1.7 MB”) plus the largest sub-collections that fit—in a measured crawl, the 15 repositories over 500 skills hold 51% of the public corpus, so the cap touches almost no repositories while defusing half the risk. *Phase 3, materialize*: sparse-checkout of the sub-path, then the leaf-rule copy—SKILL.md at the top means the whole tree is that one skill’s bundle; otherwise each skill folder lands at its true relative depth, and repository cruft (README, LICENSE, .github/), .git itself, and symlinks never land. The cap counts *skills, never files*, so a single skill with a large bundle is always allowed.

## F.2 Save = adapt + detach

---

**Algorithm 2** SAVE( $id$ ) — conflict-safe by construction, no stored state beyond two lines

---

```

1:  $dest \leftarrow .atskills/\|disk(id)$ 
2: if  $dest$  holds skills then ▷ save-again: the only “update” in the protocol
3:    $s \leftarrow$  nearest  $.source$  at or above  $dest$ 
4:   if  $s = \perp$  then refuse: it is the project’s own work; nothing is touched
5:   end if
6:   if  $\neg UNEDITED(id, dest, s.rev)$  then
7:     conflict: touch nothing; offer  $keep \cdot refetch \cdot agent$  merge with base  $s.rev$ 
8:   end if
9: end if
10:  $r \leftarrow HEADREVISION(id)$ 
11:  $FETCH(id, staging, r)$ ; atomically swap  $staging \rightarrow dest$  ▷ a failed fetch can never destroy a copy
12: write  $.source$ : line 1 =  $id$ ; line 2 = date +  $r$  ▷ written once; never resolved against

13: function UNEDITED( $id, dest, rev$ )
14:   if  $rev = \perp$  then return false ▷ unverifiable counts as edited — refusing is the safe answer
15:   end if
16:    $u \leftarrow FETCH(id, tmp, rev)$  ▷ the recorded revision is immutable and fetchable by hash
17:   return files( $u$ ) = files( $dest$ ) byte for byte
18: end function

```

---

The decision needs no digests, no staging directories, and no state beyond  $.source$ ’s two lines, because git’s content addressing already provides the merge base: “unedited” is verified against upstream *at the recorded revision*, not against a stored hash that could go stale or be forged. The asymmetry is deliberate: a copy that cannot be verified is treated as edited, which routes through the conflict path and can never silently overwrite work.

## F.3 Residency: the auto-trigger set

---

**Algorithm 3** RESIDENCY() — what fires on its own, as one prompt block

---

```

1:  $E \leftarrow$  parse  $.atskills/.autotrigger$  ▷ gitignore syntax; comments dropped; duplicates load once
2:  $R \leftarrow \emptyset$  ▷ deduplicated by skill directory: one skill loads once
3:  $P \leftarrow$  one ignore-ruleset from all plain lines ▷ so ! negation composes as in git
4: for all  $s \in walk(.atskills/)$  do if  $P$  matches  $s.rel$  then  $R \leftarrow R \cup \{s\}$ 
5: end for
6: for all cloud lines  $\ell = (@id, wholedir)$  do
7:   if  $.atskills/disk(id)$  holds skills then ▷ a saved copy answers its own @ line
8:      $R \leftarrow R \cup walk(.atskills/\|disk(id))$ 
9:   else
10:     $res \leftarrow READTHROUGHCACHE(id)$  ▷ once per session; offline serves cache, marked stale
11:    if  $res$  is a skill then  $R \leftarrow R \cup \{res\}$ 
12:    else if  $res$  is a menu and  $wholedir$  then  $R \leftarrow R \cup res$ 
13:    else report the line once; continue ▷ per-line isolation: no line breaks the session
14:    end if
15:  end if
16: end for
17: return block: one row “- name: description (path)” per  $R$ , frontmatter only ▷ bodies load on trigger

```

---



The output of Algorithm 3 is a plain string, and that is an architectural statement: the client that implements the protocol computes the block, and the host that assembles the model’s prompt splices it in verbatim. In AdaL, the TypeScript frontend runs Algorithms 1 and 3 (with the shared machine-wide cache at `~/cache/atskills`) and the Python backend contains *zero* protocol logic—it stores one string. Any agent whose prompt assembly can accept a string can adopt the protocol without touching its serving layer, and the resolution shown in the management surface can never disagree with what the model receives, because both are the same computation.

#### F.4 The checkbox toggle

---

**Algorithm 4** TOGGLE( $t$ ) — the `/skills` checkbox writes the same lines a hand edit would

---

```

1: if  $t$  is a followed cloud row then add or remove its @ line           ▷ install = the line, exactly
2: else if  $t$  is checked by its own line then remove that line
3: else if  $t$  is covered by a directory line  $D/$  then                     ▷ uncheck under a cover ⇒ split
4:   remove  $D/$ ; add explicit lines for every other skill under  $D$  that stays on
5:   ▷ sub-directories that remain whole stay as directory lines — the file reads true
6: else if  $t$  is covered by a glob pattern then refuse, naming the pattern line to edit   ▷ a pattern has no split
7: else add  $t$ ’s line; a directory row adds rel/ and removes now-covered descendant lines
8: end if
9: return a one-line note stating exactly which lines were written or removed

```

---

The toggle is the protocol’s answer to “management for people who will never hand-edit a dotfile”: every checkbox action is defined *as* an edit to `.autotrigger`, so the dialog, the typed `/skills` verbs, the @ suffixes, and a text editor are four interchangeable ways to produce the same one-line diffs, all reviewed the same way in a pull request. The split rule is the only nontrivial case, and it preserves the file’s honesty: after unchecking one skill under a covering directory line, the file names exactly the set that remains on—never a directory line that silently over-claims.

## G Discussion

Four implications follow from the analysis: what AGENTS.md’s dominance teaches, what unification buys, how author incentives change, and what the security posture is.

**Why AGENTS.md won, and what that implies.** Appendix C diagnosed the defection mechanically. The lesson is strategic. Skills lost on *delivery simplicity*, not on content quality, and any successor has to win on the same axis. We cannot tell teams that defected *from* skills apart from teams whose AGENTS.md practice simply came first, but either way delivery decided it. The three-tier model competes on that axis while fixing the costs. Tier 2 is AGENTS.md factored into pieces: the working set lives in the repository just as AGENTS.md does, but each playbook loads on demand at the best position for compliance instead of every instruction taxing every message.

**Unifying a fragmented integration layer.** One format is scattered over 54 project-level and 58 user-level directories across 75 agents, and even major providers pay per-agent packaging costs (Appendix B). The protocol unifies that layer mostly by removing the need to place anything. Referenced skills live nowhere, saved and installed skills live in one agent-neutral project home with one trigger file, and personal collections have a home in the hub. Vendor and home directories keep working as read paths. The result is one resolution model over the same files, an adoption path any agent can join at either level (Sec. E.7), and existing skills unchanged, so the per-vendor install step becomes optional rather than one more competing island.

**Author incentives: plays versus album sales.** Install counts measure the wrong thing. An install is a one-time event that says nothing about use, and under install-only delivery a skill outside each user’s small installed set is never installed at all, so the long tail reads as zero and authors stop publishing. References are plays rather than album sales. Every load is an act of use, countable per skill, and every published skill has a real chance of being played. Usage-based counting closes the incentive loop that install counts break, for the same reason streaming changed music: distribution stops being gated on shelf space.

**Alternative: retrieval-based triggering.** A harness could keep all descriptions outside the prompt, embed them, retrieve the top matches for each message, and inject those at the end of the context. That would give auto-triggering without residency, and it would also close tier 1’s discovery gap, since a user cannot reference a skill they have never heard of. Production agents already do something similar for tool schemas, loading deferred tools by search instead of keeping them resident. We see this as complementary rather than competing. Retrieval restores implicit discovery for

the long tail at the price of bringing back probabilistic activation, while explicit reference keeps loading deterministic. The `finder` skill and `:index` are our current answers, and a retrieval layer over the catalog is natural future work.

**Security considerations.** A referenced skill is instructions fetched from the network and then followed by an agent. That is remote content executed as behavior, the channel indirect prompt injection weaponizes [23, 41]. Systematic benchmarking finds no current defense reliably stops injection at the model level [24], and audits of installable LLM-extension ecosystems show they ship exploitable artifacts in practice, with 5.5% of open-source MCP servers carrying tool-poisoning vulnerabilities [25, 42–44]. Provenance and curation, not model-level filtering, have to carry the security burden.

The protocol’s properties help more than they hurt. Explicit invocation means the user knows which skill loaded and when, whereas auto-triggered skills load silently. Because content arrives as visible tool output, the full source, including every reference file, can be inspected in the transcript instead of executing unseen from a config directory. References the agent makes on its own bring implicit activation back and must appear in the transcript exactly as user references do (Fig. 3). Tier 1 leaves no installed behavior behind, though side effects of any script it runs are bounded by the session’s tool permissions, as with any agent action. On-demand delivery also allows a vetting step that install-first delivery structurally lacks. Because a reference is a single-session act, an unfamiliar skill can be trialed in a sandbox, whether an isolated worktree, a container, or a restricted-permission session, and watched end to end before it is granted any persistence. Installation instead places a skill into every future session sight unseen. Saving puts the full skill text in a git diff, open to ordinary code review, which holds the team to a reviewed copy rather than a mutable upstream, and the `.source` stamp records the exact upstream revision taken, so “did we change it or did they” is always answerable. Tier 3 activation runs through the same channel, since adding a line to `.autotrigger` is a one-line diff, so a change to the team’s resident prompt is reviewed before it reaches anyone’s session; script execution from followed skills is confirmed by revision change, never silently (Sec. E.4). Catalog slugs inherit the usual package-namespace risks such as typosquatting and name confusion, and digests and provenance metadata are the mitigations. Injection risk from malicious skill content is real but not new. It is the same for installed skills, except that an installed skill’s body loads invisibly.

**Limitations and disclosure.** This is a design paper grounded in ecosystem measurement, published-skill evidence, and the positional-attention literature. We have not yet run controlled experiments isolating trigger reliability as a function of install count, or compliance as a function of injection position, for skill-sized instructions specifically. The token measurements come from one real working setup rather than a survey. If future architectures or harness-side calibration reduce positional bias, the residency argument weakens, though the standing-tax, dilution, and lifecycle arguments do not. A referenced skill also drifts from the task in very long sessions, and re-referencing is the cheap, deterministic remedy. Tier 1 puts a network round trip on the critical path, though the validating cache reduces repeated use to one revision probe and serves the cached copy offline; a reference still resolves to living upstream content by design, so for anything the team must control the answer is a save—a local, reviewed copy—rather than a version pin the protocol deliberately does not offer. Finally, a disclosure. The author founded SylphAI, which develops AdaL and operates the catalog and hub described here, so this paper advocates infrastructure its author runs. The format and protocol are open precisely so that neither is required: @skills:gh: references and plain self-hosting work with no catalog at all.

## H Related Work

**Positional attention and long-context limits.** Liu et al. [6] established that language models attend most reliably to the beginning and end of long contexts, with systematic degradation in the middle. Subsequent work strengthened the finding: the bias is an intrinsic, U-shaped attention artifact present regardless of content relevance [37], mirrors human serial-position (primacy/recency) effects and resists prompt-level mitigation [36], and effective context is far smaller than advertised context across model families [10]. Input length alone—holding the task fixed—degrades reasoning well below nominal limits [9], and irrelevant context measurably distracts models even when they are told to ignore it [11]. This literature supplies the mechanical basis for our delivery claims (Sec. C.1).

**Instruction density and multi-turn decay.** Complementary evidence tracks what happens to *instructions* specifically: adherence falls as the number of simultaneous instructions grows, to 68% for the best frontier model at 500 constraints [12]; system-message compliance decays over conversation turns [7]; and models lose an average of 39% in long multi-turn settings versus single-turn on identical tasks [8]. In-context learning is similarly sensitive to what is in the prompt and where—example selection and ordering alone swing performance between near state-of-the-art and near chance [45, 46]. Together these justify treating resident-instruction capacity as the scarce budget of Sec. C.1; prompt-compression work [38] attacks the same cost from the other side.

**Retrieval-augmented generation.** RAG [47, 48] retrieves *declarative* knowledge at inference time rather than storing it in weights; Rubin et al. [49] extended retrieval to prompt content itself, and Self-RAG made the retrieval decision adaptive—fetch only when needed [50]. Tier-1 reference is the analogous move for *procedural* knowledge: rather than storing instructions in the prompt (the “weights” of a session), fetch them at the moment of need. The difference is the trigger: RAG retrieval is implicit and similarity-based; @skills references are explicit and deterministic, which is what reliability requires for instructions as opposed to facts.

**Agent memory and skill libraries.** Weng [51] framed the canonical agent as planning, memory, and tools, with the context window as bounded working memory. Agents that accumulate reusable text-form knowledge validate the premise that procedural text substitutes for weight updates: Voyager grows a library of executable skills and transfers it to new worlds [52]; Reflexion stores verbal reflections that improve later trials [53]; ExpeL distills cross-task insights recalled at inference [54]. Closest to our design, Agent Workflow Memory induces reusable workflows from trajectories and *selectively provides* them to the agent, improving web-task success by 24–51% relative [55]—retrieval-not-residency, demonstrated. All of these are *self-authored, per-agent* accumulations; the @skills catalog externalizes the same loop into a shared, human-curated commons with distribution.

**Tool learning at scale and agent protocols.** Toolformer [56] and ReAct [57] established models invoking external capabilities; at ecosystem scale, retrieval becomes architecturally mandatory—Gorilla pairs the model with a documentation retriever precisely because baked-in API knowledge goes stale [58], and ToolLLM equips its agent with a neural API retriever because 16,464 APIs cannot fit in context [59]. The same argument at 56,804 skills is this paper. The Model Context Protocol [4, 44] standardizes agent-to-service transport; MCP unifies how agents call *services*, @skills unifies how agents load *instructions*, and the two compose (Sec. E.6). Production harnesses likewise load deferred tool schemas by search rather than keeping them resident, the same residency-avoidance move applied to tools instead of instructions. Coding agents—the deployment surface for skills—operate in repo-scale settings where surrounding structure, not raw model capability, drives performance [60, 61].

**The skills ecosystem in practice.** Anthropic’s Agent Skills define the format this protocol delivers [2, 3]; skills.sh [1] indexes the public corpus, popularized one-command installation, and has since added per-skill selection and a use command that emits one skill as a prompt without installing it—the closest neighbour to this work, and an independent signal that on-demand delivery is where the ecosystem is heading. It differs in what we argue is the load-bearing part: a skill is selected by name inside a repository rather than addressed by path, exactly one skill resolves per invocation (a collection or subtree has no expressible form), and nothing is cached between calls, so use re-fetches while the install lifecycle it sits beside—lockfiles, per-agent directories, update and remove—remains intact underneath; the discovery convention adds self-hosted publishing with digests [34]; and AGENTS.md [35] is the competing convention whose dominance Appendix G analyzes. Practitioner analysis converges on our premises from experience rather than measurement: Willison [5] attributes skills’ promise to on-demand loading and token frugality against MCP’s tens-of-thousands-of-tokens residency; Osmani [33] demonstrates one SKILL.md corpus portable across five harnesses; Anthropic [39] prescribes just-in-time retrieval and treating context as a finite attention budget; and Karpathy [62] frames the context window as the RAM of an LLM operating system; MemGPT built exactly that virtual-memory design, paging content between context and external storage [63]. In that framing, install-only skills are programs pinned permanently in RAM, and @skills is paging.

**The @ convention.** Explicit context addressing via @-mentions is the de facto interaction standard across coding agents. AdaL [18] and Claude Code mention both individual files and whole directories with @path; Cursor’s @-symbols scope files, folders, documentation, and web results [19]; GitHub Copilot exposes @workspace [20]; and the same affordance—files, folders, symbols, URLs, terminal state—is documented in Windsurf, Continue, Cline, Sourcegraph Cody, Zed, Amazon Q, Gemini CLI, and JetBrains AI Assistant [21, 22, 64]. Across a dozen agents, @ is how users deliberately place something into the context window—yet every one of these mechanisms addresses *data*; none addresses *procedures*. @skills completes the pattern: the same gesture, applied to the one resource type that today can only be installed.

**Package management.** The install-centric skills lifecycle imports the package-manager mental model (registry, install, update), but software packages are the wrong analogy for prompt content: code costs nothing until called, while a resident description costs attention on every message. The closer analogy is the open-format-plus-hub pattern—git and GitHub, model weights and Hugging Face—where an open artifact enabled a hosted service to make management effortless, and the value accrued to distribution and curation rather than format ownership (Sec. E.8).